
PHASER GAME: PLATFORMER

2D Graphical Browser Game - (JavaScript OOP + Phaser Framework)

Table of Content: *Platformer Game in JavaScript & Phaser Framework*

Topic	Description	Page
Introduction	Learning Objectives: JavaScript OOP, Phaser framework, Single Page Apps	2
OOP Design	Develop a Roadmap: Plan Full Featureset, Identify MVP, Divide Task into Dev Cycles, Goals, & Milestones	6
Dev Cycle 1:	Goal 1: Set-up & Launch (Empty) Phaser Game Project	10
Core Version	Goal 2: Load & Draw Level Map in Scene	13
MVP Features	Goal 3: Add Player into Scene	16
	Goal 4: Move Player with Inputs & Physics	19
	Goal 5: Add Gravity to Scene	22
	Goal 6: Add Camera to Scene	24
	Goal 7: Player Collisions with Ground/Wall Tiles	26
	Goal 8: Game Over	29
Dev Cycle 2:	Enhancement 0: Setup for Interactive Item Objects	32
Full Version	Enhancement 1: Add Goal to Scene & More Levels	34
Buildout Features	Enhancement 2: Hazards -- Deadly Tiles	39
	Enhancement 3: Hazards -- Falling Tiles	44
	Enhancement 4: Collectables	47
	Enhancement 5: Animations	50
	Enhancement 6: Enemies	54
	Enhancement 7: Title Screen	61
Conclusion	Final Comments - Future Improvement - Submission - Module 2 Project	65



Lab Introduction

Introduction

In this lab, you will build a complete, classic **Platformer** game from the ground up. This isn't just about learning game development; it's about mastering core software engineering principles and advanced framework concepts in a fun, hands-on way.

You'll see firsthand how powerful concepts like Object-Oriented Programming (OOP)—especially **Inheritance**—allow you to take a professional-grade framework like Phaser and extend it to build a multi-level game with clean, reusable code. By the end, you'll have a fully functional game and a much deeper understanding of how modern JavaScript applications are built.

What You Will Build & Learn

Your goal is to build a complete **Platformer Game** that runs in a browser. Along the way, you will learn to:

- **Master Tilemaps:** Use Phaser's powerful tilemap system to build complex, multi-screen levels from data created in an external editor like Tiled.
- **Implement Platformer Physics:** Manage world gravity, character jumping, and collisions with level geometry to create a classic platforming feel.
- **Structure a Multi-Scene Project:** Organize code for a title screen and multiple game levels, and manage transitions between them.
- **Leverage Advanced Inheritance:** Extend core Phaser classes and, more importantly, extend your *own* custom classes (creating `Level2` by inheriting from `Level1`) to build new features with minimal repeated code.
- **Define Data-Driven Game Objects:** Create `Player`, `Enemy`, and other interactive items by reading their positions and properties directly from the tilemap data.
- **Implement an Agile Workflow:** Build a complex project from a simple MVP to a feature-rich final product using small, testable milestones.

Requirements

Knowledge:

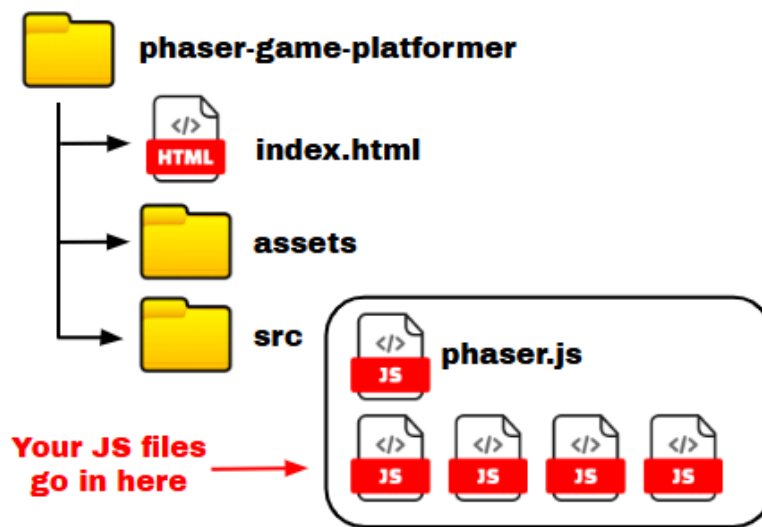
- A solid understanding of basic JavaScript.
- Familiarity with Object-Oriented Programming concepts, including:
 - Classes and Objects (instances)
 - Constructors
 - Properties and Methods
 - Inheritance (`extends`)

Tools:

- A modern web browser (Google Chrome is recommended).
- A local web server to run the project. A simple tool like `http-server` for Node.js is perfect.
- An optional (but recommended) tool to inspect the level files is the [Tiled Map Editor](#).

Project Architecture:

Start this project by downloading the starter files from GitHub. See the project structure below.



Download Starter files:

<https://github.com/scalemailed/phaser-game-platformer>

Core Concepts: How a Phaser Game Works

Before we write any code, let's establish a simple mental model of the Phaser framework. Understanding this core structure is the key to everything we'll build.

The Big Picture

Phaser organizes your game into a clear hierarchy:

- **The Game:** This is the top-level object that manages the entire application window and launches your game. We will only create this once in `game.js`.
 - **Scenes:** A game is made up of one or more Scenes. Our platformer will be a multi-scene application from the start, using a `TitleScene` for the main menu, and `Level1` and `Level2` for gameplay.
 - **Game Objects:** These are the interactive things that live inside a Scene, like the `Player`, `Enemy` characters, and items like collectibles and hazards.
 - **Tilemaps:** A tilemap is a grid-based system for efficiently creating large, complex game levels from a data file (`.json`) and a tileset image (`.png`). This is how we'll build our entire world.
-

The Scene Lifecycle: The Heart of Your Game

Every Scene in Phaser follows a critical, predictable lifecycle. You will write your own custom code for three main methods that Phaser calls automatically:

- `preload()`: Phaser calls this first. Its only job is to load all the assets your Scene needs (images, sounds, etc.) into memory before the Scene begins.
 - `create()`: Once everything is preloaded, Phaser calls this method. Here, you will set up the initial state of your Scene by creating your Game Objects (like the player and enemies) and defining things like collision rules.
 - `update()`: This method is a loop that runs on every single frame (typically 60 times per second). All of your real-time game logic - like checking for keyboard input and moving characters - will go here.
-

Inheritance: Your Code works within the Framework

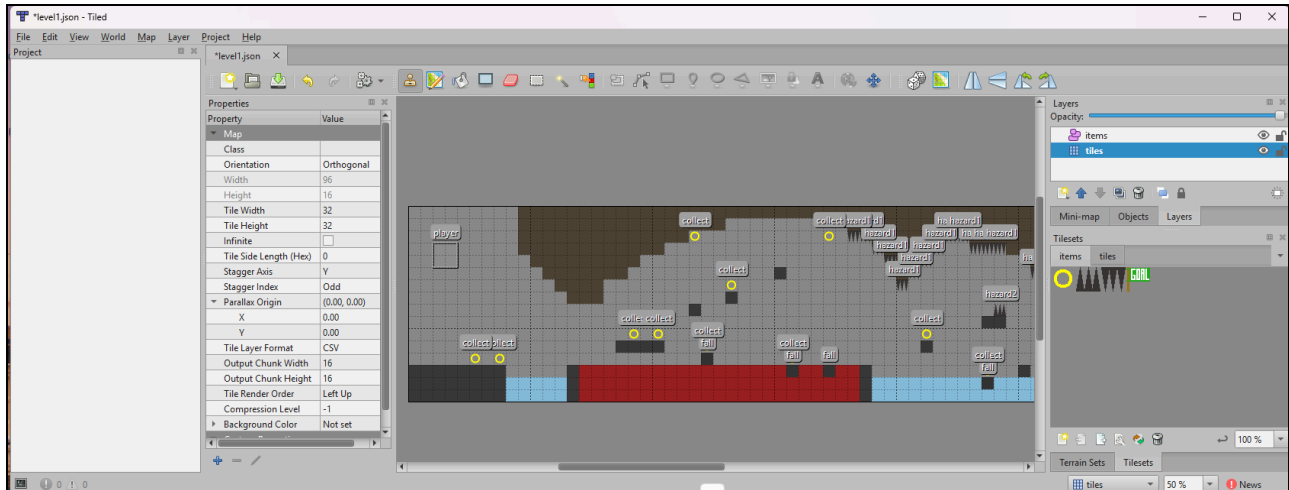
The "magic" of working with a framework like Phaser comes from **inheritance**. Instead of writing rendering and physics code from scratch, we create our own custom classes that `extend` built-in Phaser classes.

For example, our `Player` class will extend `Phaser.Physics.Arcade.Sprite`. By doing this, our `Player` object instantly "inherits" all the features of a sprite—it can be rendered, have physics applied to it, and work with the entire Phaser engine, all with just one line of code. We can additionally extend our own classes too.

Tilemaps

A tilemap is the standard, efficient way to build levels for 2D games. Instead of placing hundreds of individual images for the ground and walls, we use a specialized tool called a **Tilemap Editor** (like Tiled) to "paint" the level on a grid. This editor then exports a **blueprint** (.json file), and we provide it with a **"palette"** of graphics (.png tileset). Phaser reads this blueprint and uses the palette to construct the level.

The screenshot below shows our `level1.json` map open in the **Tiled editor**. Let's break down what we're seeing:



The properties for the entire map are saved directly into `level1.json`. You can see these in the **"Properties"** panel on the left of the screenshot.

1. Map Dimensions (The Blueprint's Size):

- **Tile Size:** The properties show a `Tile Width` and `Tile Height` of **32x32 pixels**. This is the size of each individual square graphic in our tileset.
- **Map Size:** The properties show a `Width` of **96** and a `Height` of **16**. This is the size of the entire level *in tiles*.
- **Total Pixel Size:** This explains why we temporarily set our game's width to $32 * 96$ in an earlier goal. We were making the canvas large enough to see the entire level at once for testing.

2. Tilesets (The "Palette"):

- **In the bottom right of the screenshot, you can see the "Tilesets" panel**, which lists `tiles` and `items`. The `level1.json` file contains a section called `"tilesets"` that references these image files.

- This section acts like a legend, telling Phaser: "To build this map, you will need `tiles.png` (for the ground) and `items.png` (for interactive objects)."

3. Layers (The "Canvases"):

Tilemaps are built with layers, visible in the **"Layers" panel in the top right of the screenshot**. Our `level1.json` has two important layers that work together to create our world:

- **Tile Layer (`tiles`)**

This is the main grid for the level's visual layout and geometry. The JSON file stores a giant array of numbers for this layer, where each number corresponds to a specific tile from the `tiles.png` tileset. This is the core "paint-by-numbers" part of the map.

A tile in this layer isn't just a picture; it can also have data attached to it. This is how we distinguish between a solid platform and a purely decorative background tile. In Tiled, we can assign **Custom Properties** to specific tiles within the tileset. For example, in our lab, we give all solid ground and wall tiles a property of `terrain: 'block'`. Later, in our Phaser code, we will tell the physics engine: "Make any tile with this exact property solid." This is a powerful technique that separates a tile's visual appearance from its physical behavior.

- **Object Layer (`items`)**

This layer is used to place things that don't fit perfectly on the tile grid. Instead of a grid of numbers, it contains a list of objects with precise X/Y coordinates. The most important feature of these objects is their `name` property, which we use in our code to identify and create them. Objects in this layer can be used in two distinct ways:

1. **Abstract Placeholders:** Some objects are just location markers. In Tiled, our `player` and `enemy` objects are simple rectangles that exist only to define a starting position and an identity (`"name": "player"`). Our Phaser code finds this marker and, at its location, creates a complex, custom JavaScript object (our `Player` or `Enemy` class).
2. **Concrete Tile Objects:** Other objects are placed in Tiled using an actual image from a tileset. Our `goal`, `**collect**ible`, and `hazard` objects are examples of this. They have a `name`, a position, and a specific visual representation. Our Phaser code uses a helper function (`createFromObjects`) to create simple sprites that look exactly like they do in the editor.

OOP Design in JS

'Approach' → Plan phase

Game Objective:

Our mission is to build a classic side-scrolling **Platformer** game. You will control a player who must run and jump through levels, navigate platforms, avoid deadly hazards, and stomp on enemies. We will build this game from the ground up using Object-Oriented principles to create clean, reusable code that integrates perfectly with the powerful Phaser 3 game engine and its tilemap system.

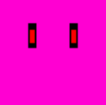
'Apply' → Do phase

Step 1: Define the Core Gameplay & Feature

First, let's identify the key objects in our game from the player's point of view. We'll split the features for each object into a Minimal Viable Product (MVP) - the simplest playable version - and the Full feature set (our "Enhancements") we'll build out later.

Game Objects (from Player Perspective):

Characters

	Player character {name: 'player'} (MVP) - Can move left, right, and jump. Collides with the ground. (FULL) - Has a walking animation. Can stomp on enemies to destroy them.
	Enemy character {name: 'enemy'} (FULL) - Moves back and forth on platforms. Respawns if it falls into void. Kills the player if they touch it. Can be destroyed by jumping on top of it.

Tiles

	Tile - Land {terrain: 'block'} (MVP) - Solid tiles that form the level's geometry. The player can stand on them.
	Tile - Stone {terrain: 'block'} (MVP) - Solid tiles that form the level's geometry. The player can stand on them.
	Tile - Water (MVP) - A visual tiles that fills voids to appear as water. The player passes through them.
	Tile - Lava (MVP) - A visual tiles that fills voids to appear as lava. The player passes through them.

Items

	Item - Goal {name: 'goal'} (FULL) - An item that loads the next level of the game when the player touches it.
	Item - Collectable {name: 'collect'} (FULL) - An item the player can collect, which disappears on contact.
	Item - Hazard - Spikes Up {name: 'hazard1'} (FULL) - A static tile that restarts the level if the player touches it.
	Item - Hazard - Spikes Down {name: 'hazard2'} (FULL) - A static tile that restarts the level if the player touches it.
	Item - Hazard - Falling Tile {name: 'fall1'} (FULL) - A platform tile that appears solid but falls shortly after the player touches it

Step 2: Map Gameplay Behavior to JS Classes

Next, let's translate those game concepts into the actual JavaScript classes and files we will create. This is our technical blueprint.

Overview of Classes/Files (from Developer's Perspective):

Class/File	Responsibility
game	<i>The entry point. This script creates the main Phaser Game configuration and tells it which Scene to launch first.</i>
TitleScene	<i>A simple scene that displays the game's title and instructions. Acts as the starting point for the game.</i>
Level1	<i>The heart of our game. This class will manage all the game objects, handle spawning, process collisions, and contain the main game loop (<i>preload</i>, <i>create</i>, <i>update</i>).</i>
Level2	<i>A second level that inherits from Level1, demonstrating how to reuse code and override methods for new content.</i>
Player	<i>Defines the Player character. This class will handle keyboard input for movement and jumping.</i>
Enemy	<i>Defines a single Enemy. This class will manage its back-and-forth movement and interaction with the player.</i>

\

Step 3: Define Our Development Roadmap

A key principle of modern software development is to build iteratively. We won't try to build everything at once. Instead, we'll first build the **MVP (Goals 1-8)** to get a simple, playable game. After that, we will add the remaining features as a series of **Enhancements**.

This roadmap will be our guide for the entire lab.

MVP Features - Core Build - Roadmap	Buildout features - Full Build - Roadmap
Goal 1: Set-up & Launch (Empty) Phaser Game Project Goal 2: Load & Draw Level Map in Scene Goal 3: Add Player into Scene Goal 4: Move Player with Inputs & Physics Goal 5: Add Gravity to Scene Goal 6: Add Camera to Scene Goal 7: Player Collisions with Ground/Wall Tiles Goal 8: Game Over	Enhancement 0: Setup for Interactive Item Objects Enhancement 1: Add Goal to Scene & More Levels Enhancement 2: Hazards -- Deadly Tiles Enhancement 3: Hazards -- Falling Tiles Enhancement 4: Collectables Enhancement 5: Animations Enhancement 6: Enemies Enhancement 7: Title Screen

Dev Cycle 1:

Core Version - MVP Features

Part 1: Building the Minimal Viable Product (MVP)

It's time to write some code. In this first major section of the lab, we will build a complete **Minimal Viable Product (MVP)** for our platformer game from scratch.

The goal here is to create a barebones but fully playable version of our game. This is a core principle of agile development: build a simple, working foundation first, then add the buildout features (the "enhancements") later.

Your Objective for Part 1

By the end of this section, you will have a simple but operational **platformer game**. The final result will include:

- A game scene that renders a multi-screen level built from a **Tilemap file**.
- A player character who is affected by **gravity** and can **collide** with the ground and walls.
- Player controls for running **left, right, and jumping** with keyboard input.
- A **camera** that follows the player as they navigate the level.
- A complete game loop: **falling off the stage** will trigger a "Game Over" and restart the level.

Table of Contents

Goal 1: Set-up & Launch (Empty) Phaser Game Project	10
Goal 2: Load & Draw Level Map in Scene	13
Goal 3: Add Player into Scene	16
Goal 4: Move Player with Inputs & Physics	19
Goal 5: Add Gravity to Scene	22
Goal 6: Add Camera to Scene	24
Goal 7: Player Collisions with Ground/Wall Tiles	26
Goal 8: Game Over	29

Goal 1: Set-up & Launch (*Empty*) Phaser Game Project

'Approach' → Plan phase

Our first step is to create the basic file and folder structure for our project. This foundation will hold all our code and assets.

The project root directory will contain.

- **assets/**: directory to hold all of the game's art
- **src/**: directory to hold all of the game's code
- **index.html**: file that launches your game in the browser

Goal: Successfully set up the project files and launch an empty, black 768x512 Phaser game window.

API References:

- <https://photonstorm.github.io/phaser3-docs/Phaser.Game.html>

'Apply' → Do phase

Step 1: Create the index.html File

Since Phaser runs in a browser, we need an `index.html` file to act as the container for our game. This file will load the Phaser framework and all of our custom JavaScript files.

For now, we will create the `index.html` file and include the three scripts essential for starting the game:

- `phaser.js`: The Phaser 3 game framework library.
- `Level1.js`: Our class file for the first game scene.
- `game.js`: The script that configures and launches the game.

Important: The order of the `<script>` tags is critical. A script cannot use code from another script unless the latter has already been loaded. Therefore:

1. `phaser.js` must come **first** because everything depends on it.
2. `Level1.js` must be loaded **before** `game.js` because `game.js` will tell Phaser to start the `Level1` scene.

index.html

```
<html>
  <head>
    <script src="./src/phaser.js">    </script>
    <script src="./src/Level1.js">    </script>
    <script src="./src/game.js">     </script>
  </head>
  <body></body>
</html>
```

Step 2: Create the Main Game Configuration (game.js)

Next, create a `game.js` file inside the `src/` directory. This is the entry point of our game. It defines a configuration (`config`) object with the game's core settings and then uses it to create a new `Phaser.Game` instance, which starts the game.

- `type`: Tells Phaser to use the Canvas API for rendering.
- `scene`: An array with the game's scenes. The first scene in the array (`Level1`) will load on startup.
- `pixelArt`: This setting optimizes rendering for pixel-art graphics.

`src/` → `game.js`

```
const config = new Object();

config.type = Phaser.CANVAS;           //HTML Rendering API
config.scene = [ Level1 ];             //Scenes in this game

const game = new Phaser.Game(config);   //Start game with these configs
```

Step 3: Create the Level1.js Class

Now, create the `Level1.js` file. This will be our first playable level. For now, we'll create a "stub" or "skeleton" class that extends `Phaser.Scene`. This makes it a valid scene that Phaser can manage and includes the core methods of the scene lifecycle.

A Phaser Scene has a predictable lifecycle with several key methods that are called automatically:

- `constructor()`: Called once when the scene is first created.
- `preload()`: Used to load all necessary assets (images, audio, tilemaps).
- `create()`: Used to set up the initial state of the scene (create the player, enemies, etc.).
- `update()`: The main game loop, called on every frame.

The `super('Level1')` call is essential. It calls the constructor of the parent class (`Phaser.Scene`) and registers this scene with Phaser's Scene Manager using the unique key `'Level1'`. We use this key in `game.js` to identify which scene to start.

`src/` → `Level1.js`

```
class Level1 extends Phaser.Scene {

    //construct new scene
    constructor(key='Level1') {
        super(key);    //set this scene's id within superclass constructor
    }

    //preload external game assets
    preload() { }

    //create game data
    create() { }

    //Update game data
    update() { }
}
```

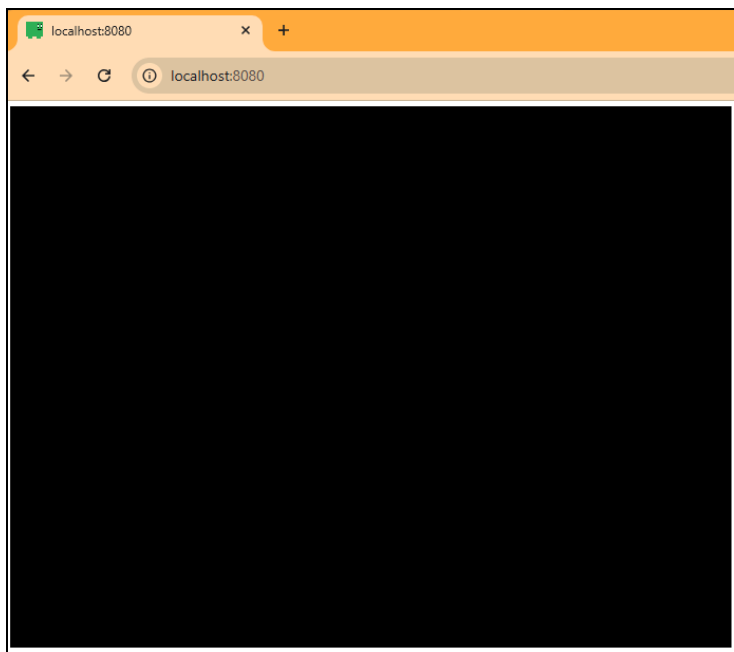
'Assess' → Test phase

Test Instructions:

1. Ensure you have `phaser.js`, `game.js`, and `Level1.js` inside your `src` folder.
2. Launch an HTTP server to host the `index.html` file from the project's root directory.
3. Open the browser to the specified URL (e.g., `localhost:8080`).

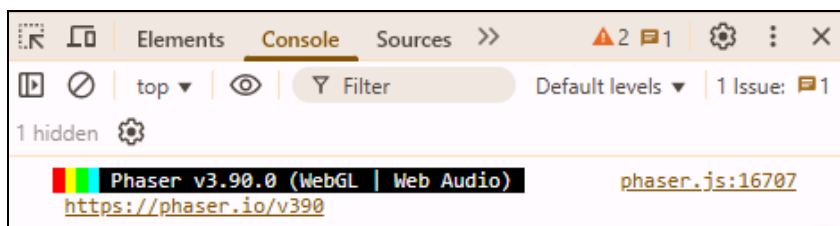
Expected Output (Viewport):

A 768x512 resolution black screen should render in the viewport, successfully launching an empty Phaser game.



Expected Output (Devtools Console):

The console should display that Phaser is successfully launched with its version number and rendering settings.



Goal 2: Load & Draw Level Map in Scene

'Approach' → Plan phase

Now that our project is set up, let's render our first level. In modern 2D games, levels are rarely built by placing individual sprites. Instead, we use a **Tilemap**. This is a more efficient method where a level is designed in an external editor (like [Tiled](#)) and exported as data (a `.json` file) and a tileset image (`.png`). Phaser then reads this data to construct the level for us.

Understanding Tilemaps

- **Tileset:** A single image file (`tiles.png` in our case) that contains all the small, square graphics (like grass, dirt, blocks) used to build the level.
- **JSON Data:** A text file (`level1.json`) that acts as a blueprint. It contains information about the map's dimensions and, most importantly, a grid that tells the game which tile from the tileset goes into each position of the level.

In this goal, we will load these two assets and tell Phaser how to draw the level based on them.

API References:

- [Phaser.Loader.LoaderPlugin#tilemapTiledJSON](#)
- [Phaser.GameObjects.GameObjectCreator#tilemap](#)

'Apply' → Do phase

Step 1: Expand the Game View for Map Testing

Before we load our map, let's adjust the game's configuration. Our `level1.json` file defines a world that is 96 tiles wide, which is much larger than the viewport we originally planned.

To verify that our entire map loads correctly, we will temporarily set our game's `width` to match the full width of this level. This creates a very wide canvas that allows us to see the whole map at once, confirming everything is working before we implement a camera.

src/ → *game.js*

```
const config = new Object();

config.type = Phaser.CANVAS;           //HTML Rendering API
config.width = 32 * 96;                 //32px/tile * 96 tiles/world
config.height = 32 * 16;                //32px/tile * 16 tiles/world
config.pixelArt = true;                 //optimized for pixel art
config.scene = [ Level1 ];              //Scenes in this game

const game = new Phaser.Game(config);    //Start game with these configs
```

Step 2: Refactor Constructor to include references to level data

We'll add properties to the class constructor to store the unique key for our map and the filename of its JSON data.

src/ → Level1.js → constructor()

```
constructor(key='Level1') {
  super(key); //set this scene's id within superclass constructor
  this.map_key = 'map1';
  this.map_json = 'level1.json';
}
```

Step 3: Preload the Tilemap and Tilesheet Spritesheet

Inside the `preload()` method, we need to load the assets required to build our map. This involves two steps:

1. `this.load.tilemapTiledJSON()`: This function loads the level's blueprint from the `.json` file. We give it a unique key (`this.map_key`) so we can reference it later.
2. `this.load.spritesheet()`: This function loads the `tiles.png` image. We load it as a "spritesheet" because it's a single image containing multiple, consistently-sized frames (our 32x32 tiles).

Phaser will not proceed to the `create()` method until all assets in the `preload()` queue have finished loading.

src/ → Level1.js → preload()

```
preload(){
  this.load.path = 'assets/'; //Define file path
  this.load.tilemapTiledJSON( this.map_key, this.map_json); //Load JSON file
  const tile_size = {frameWidth: 32, frameHeight: 32}
  this.load.spritesheet('tiles', 'tiles.png', tile_size); //Load tile spritesheet
}
```

Step 4: create() Delegates to Helper Methods to Make the Level Map

As our game grows, the `create()` method can become very cluttered. It's a good practice to organize creation logic into smaller, well-named helper methods. We'll create a `create_map()` helper to handle setting up our level.

First, add the call to this new helper inside your main `create()` method.

src/ → Level1.js → create()

```
//Create Game World
create(){
  this.create_map(); // create level
}
```


Step 5: Implement the create_map Method

Now, implement the `create_map()` helper method. This is where the magic happens. We'll convert the loaded data into a visible, interactive level in three steps:

1. **Create the Tilemap Object:** We use `this.make.tilemap()` to create a map object in Phaser's memory from our preloaded JSON data.
2. **Add the Tileset Image:** We use `this.map.addTilesetImage()` to link the visual tiles from our 'tiles' spritesheet to the data within the map object. The first argument ('tiles') is the name given to the tileset inside the Tiled editor, and the second is the key we used to load the image in `preload()`.
3. **Create the Layer:** A tilemap can have multiple layers (e.g., background, main ground, foreground). `this.map.createLayer()` takes a layer name from the Tiled file ('tiles'), the tileset object, and an x/y position to finally render that layer onto the screen.

Level1.js → create_map()

```
//Load level
create_map() {
  this.map = this.make.tilemap({ key: this.map_key }); //setup map object from tilemap

  // Arguments: 1: Tiled tileset name, 2: image key you loaded
  const groundTiles = this.map.addTilesetImage('tiles', 'tiles');

  // Arguments: 1:<Tiled layer name or index>, 2:Tileset, 3:<x offset px>, 4:<y offset px>
  this.groundLayer = this.map.createLayer('tiles', groundTiles, 0, 0);
}
```

'Assess' → Test phase

Testing:

1. Make sure you have a `level1.json` file and `tile.png` inside your `assets/` directory.
2. Launch your local server and open the game in your browser.

Expected Output (Viewport):

You should now see your game world rendered inside the canvas. The static level, built from the tileset, should be visible. Note: Depending on your map's size, you may need to **'Zoom Out'** your browser view to see the entire map at once.



Goal 3: Add Player into Scene

'Approach' → Plan phase

Let's add our first interactive game object: the **Player**. The player will be a visible sprite that can participate in Phaser's physics system for movement and collisions.

To achieve this, we will create a custom `Player` class that inherits from `Phaser.Physics.Arcade.Sprite`. This gives our player all the built-in rendering and physics capabilities of a standard Phaser sprite, which we can then extend with our own custom properties and behaviors, like how to read its starting position from the tilemap.

'Apply' → Do phase

Step 1: Preload the Player Image

First, we need to load the image asset for our player. For the MVP, we will use a single, static image. In a later development cycle, we will replace this with a spritesheet for animation.

src/ → *Level1.js* → *preload()*

```
preload(){
  this.load.path = 'assets/'; //Define file path
  this.load.tilemapTiledJSON( this.map_key, this.map_json); //Load JSON file
  const tile_size = {frameWidth: 32, frameHeight: 32}
  this.load.spritesheet('tiles', 'tiles.png', tile_size); //Load tile spritesheet
  this.load.image( 'player', 'player.png'); //load player image
}
```

Step 2: Create the Player.js Class

Next, create a new file at `Player.js`. This class will define our player object. The constructor is the most important part, as it's responsible for creating the player and setting its initial state based on data from our tilemap.

Let's break down the constructor:

- **extends Phaser.Physics.Arcade.Sprite**: Establishes that our `Player` is a physics-enabled sprite, inheriting all its features.
- **super(scene, 0, 0, 'player')**: Calls the parent constructor. We use `(0,0)` as a placeholder position because we will immediately set the correct coordinates from the map data. The `'player'` key tells Phaser to use the `player.png` image we loaded.
- **scene.map.findObject(...)**: This powerful method searches the "items" object layer in our tilemap for an object named "player" and returns its data (including its x and y position).
- **scene.add.existing(this)**: This is a crucial step. It tells the scene to add this newly created Player object to its display list, making it visible.
- **scene.physics.add.existing(this)**: This gives our Player a physics "body", allowing it to be affected by gravity and collide with other objects. This is essential for the next goal.

src/ → *Player.js*

```

class Player extends Phaser.Physics.Arcade.Sprite {
  constructor(scene) {
    super(scene, 0, 0, 'player');

    const start = scene.map.findObject('items', obj => obj.name === 'player' );
    this.x = start.x;
    this.y = start.y ;
    this.setOrigin(0.5,1);
    this.depth = 1;
    this.speed = 200;

    scene.add.existing(this);
  }
}

```

Step 3: Add a create_player Helper Method

In `Level1.js`, we'll continue our pattern of using helper methods. Add a call to `this.create_player()` within your `create()` method.

src/ → *Level1.js* → *create()*

```

//Create Game World
create(){
  this.create_map();           //helper method: create map
  this.create_player();        //helper method: create player
}

```

Step 4: Instantiate the Player

Now, implement the `create_player()` helper method. Inside it, we create a new instance of our `Player` class. We store it in a scene property, `this.player`, so we can easily access it later. Notice we pass `this` (a reference to the `Level1` instance itself) to the `Player` constructor, which is required by its `constructor(scene)` signature.

src/ → *Level1.js* → *create_player()*

```

//Create Game World
create_player() {
  this.player = new Player(this); //create player
}

```

Step 5: Load the New Player.js File

Finally, we need to load our new `Player.js` file in `index.html`. The order is very important. Since `Level1.js` creates a new `Player()`, the `Player.js` file **must be loaded before** `Level1.js`.

index.html

```
<html>
  <head>
    <script src="./src/phaser.js">    </script>
    <script src="./src/Player.js">    </script>
    <script src="./src/Level1.js">    </script>
    <script src="./src/game.js">      </script>
  </head>
  <body></body>
</html>
```

'Assess' → Test phase**Test Instructions:**

1. Make sure you have a `player.png` file inside your `assets/` directory.
2. Launch your local server and open the game in your browser.

Expected Output (Viewport):

The player sprite should now be rendered on the screen at its designated starting position from the tilemap, appearing on top of the level geometry. The player will not move or fall yet.



Goal 4: Move Player with Inputs & Physics

'Approach' → Plan phase

Let's make our game interactive by allowing the player to move. We'll use the arrow keys for input and leverage Phaser's built-in **Arcade Physics** engine to handle the movement.

First, we'll enable the physics system for our entire game. Then, we'll give our `Player` object a physics "body," which allows the engine to control its position via **velocity**. Velocity is a vector with x and y components. A positive x-velocity moves the object right, negative moves it left. A positive y-velocity moves it down, and a negative y-velocity moves it up. By changing the player's velocity based on keyboard input, we can bring it to life.

'Apply' → Do phase

Step 1: Enable Arcade Physics

In `src/game.js`, we need to add a `physics` property to our main game configuration. This tells Phaser to start the Arcade Physics engine and make it available to all scenes. Without this, our player cannot have a physics body or velocity.

src/ → *game.js*

```
const config = new Object();

config.type = Phaser.CANVAS;           //HTML Rendering API
config.width = 32 * 96;                 //32px/tile * 96 tiles/world
config.height = 32 * 16;                //32px/tile * 16 tiles/world
config.pixelArt = true;                 //optimized for pixel art
config.scene = [ Level1 ];              //Scenes in this game
config.physics = { default: 'arcade' };  //Physics: collisions & gravity

const game = new Phaser.Game(config);    //Start game with these configs
```

Step 2: Update the Player to Handle Input

Now, let's update the `Player` constructor.

- `scene.physics.add.existing(this)`: This gives our `Player` sprite a physics body, allowing it to have properties like velocity.
- `scene.input.keyboard.addKeys(...)`: We use the scene's input manager to create a cursor keys object that we can poll to check if the arrow keys are being pressed. We store this in `this.buttons` for easy access.

src/ → *Player.js* → *constructor()*

```

constructor(scene){
    super(scene, 0, 0, 'player');

    const start = scene.map.findObject('items', obj=> obj.name === 'start');
    this.x = start.x;
    this.y = start.y;
    this.setOrigin(1,1);
    this.depth = 1;
    this.speed = 200;

    scene.add.existing(this);
    scene.physics.add.existing(this);
    this.arrow_keys = scene.input.keyboard.addKeys('up,left,right');
}

```

Step 3: Create the Player's move Method

This new method in the `Player` class will contain all of its movement logic. This is a great example of **encapsulation** -- the `Player` object is responsible for knowing how it should move. The logic follows a simple "reset and check" pattern on every frame:

1. First, set the player's velocity to zero. This ensures the player stops moving if no key is pressed.
2. Then, check each arrow key. If a key is `isDown`, set the velocity accordingly.

src/ → *Player.js* → *move()*

```

//move player
move() {
    //verify that player has a physics body to move (for multilevel)
    if (this.body === undefined)
        return;

    // reset velocity
    this.body.velocity.x = 0;
    // take care of character movement
    if ( this.arrow_keys.up.isDown ) {
        this.body.velocity.y = -this.speed * 2;
    }
    if ( this.arrow_keys.left.isDown ) {
        this.body.velocity.x = -this.speed;
    }
    if ( this.arrow_keys.right.isDown ) {
        this.body.velocity.x = this.speed;
    }
}

```

Step 4: Call the Player's Logic from the Scene's update Loop

The `Level1` scene acts as the "orchestra conductor." Its main `update()` loop is responsible for telling all the game objects what to do on each frame.

First, add a call to a new helper method, `update_player()`, from the main `update()` loop in `Level1.js`.

src/ → Level1.js → update()

```
//update game state
update() {
  this.update_player();
}
```

Step 5: Implement the update_player Helper

Finally, implement the `update_player()` helper. Its only job is to call the `move()` method on our player instance. This delegates the responsibility of movement from the scene to the player itself.

src/ → Level1.js → update()

```
//update game state
update_player() {
  this.player.move();
}
```

'Assess' → Test phase

Testing:

- The player object should remain stationary when no arrow button is pressed.
- The player object should move left and right when the corresponding arrow buttons are pressed.
- The player object should stop moving horizontally after the left or right arrow button is released.
- When you press the up-arrow button, the player object should jump up.

Note: No gravity has been applied yet. Watch the player's behavior carefully:

- When you move left or right and release the key, the player stops moving horizontally. This is because we reset `this.body.velocity.x = 0;` on every frame.
- However, if you tap the **up-arrow key even once**, the player will start moving upwards and will **not** come back down. Because `velocity.y` is never reset to zero in our code, the player will continue to float up indefinitely, even after you release the key.

This demonstrates the need for a constant downward force (gravity) to counteract the upward velocity of a jump. We will add gravity and collisions to fix this in the next goals.

Goal 5: Add Gravity to Scene

'Approach' → Plan phase

In the previous goal, we discovered that pressing the up-arrow key sends our player flying upwards indefinitely. This is because there is no force to counteract the upward velocity we are applying.

Let's fix this by adding **gravity**. We will apply a constant downward force to the entire scene's physics world. This force will pull on every physics object, including our player, making them fall realistically and providing the foundation for proper jumping.

'Apply' → Do phase

Step 1: Invoke the Gravity Helper Method

As with our other setup logic, we will keep the `create()` method clean by calling a dedicated helper function for setting up gravity. Add the following line to the `create()` method in `Level1.js`.

src/ → *Level1.js* → *create()*

```
create() {  
  this.create_map();           // create level  
  this.create_player();        // create player  
  this.create_gravity();       // create gravity  
}
```

Step 2: Implement `create_gravity` method

Now, implement the `create_gravity()` method in `Level1.js`. This method accesses the scene's physics world and sets the `y` property of its `gravity`.

- A positive value for `gravity.y` creates a downward pull.
- The value `600` is a good starting point for a platformer, but you can experiment with larger or smaller numbers to make the player feel "heavier" or "lighter."

src/ → *Level1.js* → *create_gravity()*

```
//setup gravity with physics  
create_gravity() {  
  this.physics.world.gravity.y = 600;  
}
```


'Assess' → Test phase

Testing:

Launch the game and observe the new player physics:

- The player should immediately start falling from their starting position.
- When you press and hold the **up-arrow**, the player will move upwards. The powerful upward velocity from your input (-400, calculated as `-this.speed * 2`) is now fighting against the constant downward pull of gravity (600).
- When you release the **up-arrow**, the player will immediately start to fall back down.
- You can still move left and right while rising or falling.

You will notice the player still falls right through the tilemap floor. This is expected! We have added gravity, but we have not yet told the player and the ground how to *collide* with each other.

Note: This new control scheme, where you can press the jump key at any time (even in mid-air) to apply an upward force, is very similar to a "Flappy Bird" style game..

Goal 6: Add Camera to Scene

'Approach' → Plan phase

Our game world (96 tiles wide) is much larger than a typical screen, and our player can now fall due to gravity. To solve this, we will use Phaser's built-in **Camera** system. We will configure the main camera to lock onto the player and follow them as they move through the level. We will also set the camera's "bounds" to match the size of our tilemap, preventing it from scrolling into the empty void beyond the level's edges. Finally, with the camera in place, we will resize the game window back to a standard viewport size.

'Apply' → Do phase

Step 1: Refactor Game's Width for Viewport Size

In a previous goal, we set our game's width to be the full size of the map for testing. Now that we are implementing a scrolling camera, it's time to set the actual viewport size that the player will see.

Open `src/game.js` and change `config.width` back to a more standard screen width, like 24 tiles wide (32 * 24).

src/ → *game.js*

```
const config = new Object();

config.type = Phaser.CANVAS;           //HTML Rendering API
config.width = 32 * 24;                 //32px/tile * 24 tiles/screen
config.height = 32 * 16;                //32px/tile * 16 tiles/screen
config.pixelArt = true;                 //optimized for pixel art
config.scene = [ Level1 ];              //Scenes in this game
config.physics = { default: 'arcade' };  //Physics: collisions & gravity

const game = new Phaser.Game(config);   //Start game with these configs
```

Step 2: Invoke the create_camera Helper Method

Following our established pattern, we will add a call to a new `create_camera()` helper method within the `create()` method of `Level1.js`.

src/ → *Level1.js* → *create()*

```
create() {
  this.create_map();           // create level
  this.create_player();        // create player
  this.create_gravity();       // create gravity
  this.create_camera();        // create camera
}
```

Step 3: Implement create_camera method

Now, create the `create_camera()` method in `Level1.js`. This method will configure the scene's main camera.

Let's break down each line:

1. `startFollow(this.player)`: Tells the camera to automatically center itself on the `this.player` object every frame. *Test it now: the camera should follow the player, but it will scroll indefinitely, showing empty black space beyond the map.*
2. `setBounds(...)`: It prevents the camera from scrolling past the edges of the level. We use `this.map.widthInPixels` to get the exact size of our tilemap, ensuring the camera stops precisely at the world's boundaries. *Test it now: the camera should stop scrolling when it reaches the edges of the map.*
3. `setBackgroundColor(...)`: A nice bit of polish. This sets the color of any area the camera might show that is not covered by the tilemap (often called the "void"). *Test it now: the empty space should now be light blue instead of black.*

src/ → Level1.js → setup_camera()

```
//Setup camera to follow player
create_camera() {
  this.cameras.main.startFollow(this.player);
  this.cameras.main.setBounds(0,0,this.map.widthInPixels, this.map.heightInPixels);
  this.cameras.main.setBackgroundColor('rgb(204, 207, 255)');
}
```

*Implement these three statements one-by-one and test the game each time in between

A good practice is to implement these three lines one-by-one and test the game after adding each one. This helps you see exactly what effect each statement has.

'Assess' → Test phase

Testing:

Launch the game and verify the new camera behaviors.

- The viewport is now smaller, showing only a portion of the level's **width**.
- As the player moves **left and right**, the camera should smoothly follow them, keeping them centered horizontally.
- When the player approaches the left or right edges of the level, the camera will stop scrolling horizontally.

Key Observation & What's Next:

Notice the camera's **vertical** behavior. Because our game's height (`config.height`) is the same as our map's height (`this.map.heightInPixels`), the camera has no room to scroll up or down. It is effectively fixed in place vertically.

This means when the player falls due to gravity, they **will fall off the bottom of the screen**, and the camera will **not** follow them.

Goal 7: Player Collisions with Ground/Wall Tiles

'Approach' → Plan phase

Now that the player moves and the camera follows, we can address the most pressing issue: the player falls right through the world. We need to make the ground and walls solid.

We will do this using Phaser's physics engine and a powerful feature of the Tiled editor: **Custom Properties**. Instead of telling Phaser which specific tile images are solid (e.g., "tile number 3 is solid"), we can assign a property (like `terrain: 'block'`) to our tiles in Tiled. This is a much more flexible and maintainable way to design levels, as we can change the look of our solid tiles without having to rewrite any code.

Finally, we'll fix our "Flappy Bird" jump by only allowing the player to jump when they are standing on a solid surface.

'Apply' → Do phase

Step 1: Define Collidable Tiles in the Map

First, we need to tell Phaser which tiles in our `groundLayer` should be treated as solid objects. We do this by referencing the custom property we set in the Tiled editor.

- `setCollisionByProperty()`: This function scans the entire tile layer and enables the physics body for any tile that has the custom property `terrain` set to the value `'block'`. These tiles are now physically solid.

src/ → *Level1.js* → *create_map()*

```
create_map() {
  this.map = this.make.tilemap({ key: this.map_key }); //setup map object from tilemap

  // Arguments: 1: Tiled tileset name, 2: image key you loaded
  const groundTiles = this.map.addTilesetImage('tiles', 'tiles');

  // Arguments: 1:<Tiled layer name or index>, 2:Tileset, 3:<x offset px>, 4:<y offset px>
  this.groundLayer = this.map.createLayer('tiles', groundTiles, 0, 0);

  //collisions based on tile type in custom properties
  const ground_block = { terrain: 'block' };
  this.groundLayer.setCollisionByProperty( ground_block );
}
```

Step 2: Invoke helper method to create collisions

Invoke a new helper method, `create_collisions()`, from your main `create()` method in `Level1.js`.

src/ → Level1.js → create_collisions()

```

create() {
  this.create_map();           // create level
  this.create_player();        // create player
  this.create_gravity();        // create gravity
  this.create_camera();        // create camera
  this.create_collisions();     // create collisions
}

```

Step 3: Create the Collision Interaction

Tell the physics engine to make the `player` and the `groundLayer` actually collide with each other.

- `this.physics.add.collider()`: This is a core physics function. It establishes a rule between two objects. In this case, it tells Phaser: "Constantly check for overlaps between `this.player` and `this.groundLayer`. If they touch, prevent them from passing through each other."

src/ → Level1.js → create_collisions()

```

//setups collisions with physics
create_collisions(){
  this.physics.add.collider( this.player, this.groundLayer );
}

```

Step 4: Refine the Jump Mechanic

With collisions in place, the player can now stand on the ground. This allows us to fix the infinite jump problem. We will change the jump condition so the player can only jump if the bottom of their physics body is touching a solid object.

- `&& this.body.blocked.down`: This condition is now added to the jump check. `this.body.blocked.down` is a physics property that is only `true` when the bottom of the sprite is pressed against a solid object. This prevents the player from jumping in mid-air.

src/ → Player.js → move()

```

//move player
move() {
  //verify that player has a physics body to move (for multilevel)
  if (this.body === undefined)
    return;

  // reset velocity
  this.body.velocity.x = 0;
  // take care of character movement
  if ( this.arrow_keys.up.isDown && this.body.blocked.down ) {
    this.body.velocity.y = -this.speed * 2;
  }
  if ( this.arrow_keys.left.isDown ) {
    this.body.velocity.x = -this.speed;
  }
  if ( this.arrow_keys.right.isDown ) {
    this.body.velocity.x = this.speed;
  }
}

```

'Assess' → Test phase

Testing:

This is the moment of truth! Launch the game and test the core platformer mechanics.

- The player should now land on the ground instead of falling through it.
- You should be able to walk left and right along the ground and up slopes.
- If you walk into a wall, the player should stop.
- Press the up-arrow to jump. The player should jump into the air and fall back to the ground.
- **Most importantly:** Try pressing the up-arrow again while the player is in mid-air. They should **NOT** be able to jump again until they land.

Goal 8: Game Over

'Approach' → Plan phase

Our game now has a playable character who can run and jump on solid ground. However, there are no stakes or consequences. If the player makes a mistake, nothing happens.

Let's add a basic "lose condition." The most common way to lose in a platformer is to fall off the stage. We will implement a rule that if the player falls into a pit or off the bottom of the level, the scene will restart.

'Apply' → Do phase

Step 1: Check for the Lose Condition on Every Frame

The game needs to constantly monitor the player's status. The perfect place for this check is in the main `update()` loop, which runs on every single frame. We will follow our pattern of creating a dedicated helper method for this logic.

src/ → Level1.js → update()

```
//update game state
update(){
  this.player_update();
  this.game_over();
}
```

Step 2: Implement the game_over Method

Now, create the `game_over()` method in `Level1.js`. This function will contain the logic to determine if the player has lost. The logic is simple: we compare the player's vertical position (`y` coordinate) to the total height of the map.

src/ → Level1.js → game_over()

```
//check player lose conditions
game_over() {
  if (this.player.y > this.map.heightInPixels) { //check if player is lower than level
    this.scene.restart();
  }
}
```

'Assess' → Test phase

Testing:

Now it's time to test our new lose condition.

1. Launch the game.
2. Navigate your player to a pit or the edge of a platform.
3. Walk or jump off the edge, falling towards the bottom of the screen.

Expected Outcome:

The moment the player sprite's origin point goes below the bottom edge of the level, the scene should instantly restart. The player will reappear at their original starting position, ready to try again.

Dev Cycle 2:

Full Version - Buildout Features

Full Version Goal:

Congratulations on building a complete Minimal Viable Product! You now have a solid, working foundation for a platformer game. In this next section, we will transform that simple MVP into a polished and exciting game by adding a series of "enhancements." These are the features that will add depth, challenge, and visual flair to make your game feel complete.

Our Enhancement Roadmap

We will add the following features, building upon the MVP you just completed. With the exception of Enhancement 0, these modules are designed to be independent and can be completed in any order.

Enhancement 0: Setup for Interactive Item Objects	32
Enhancement 1: Add Goal to Scene & More Levels	34
Enhancement 2: Hazards -- Deadly Tiles	39
Enhancement 3: Hazards -- Falling Tiles	44
Enhancement 4: Collectables	47
Enhancement 5: Animations	50
Enhancement 6: Enemies	54
Enhancement 7: Title Screen	61

(**Note:** All subsequent enhancements (1-7) depend on the foundational code added in Enhancement 0. It is highly recommended to complete Enhancement 0 first.)

An Important Note on Applying Changes

Each enhancement guide will show you the new code to **add or change**, starting from the completed MVP. However, as you complete each enhancement, you are modifying your new, feature-rich codebase.

Be careful not to accidentally delete or overwrite the code you added in a previous enhancement. For example, when you start Enhancement 2, make sure you don't remove the hitbox code you added in Enhancement 1.

The Final Result (*Deliverable*)

By the end of this section, you will have a complete, feature-rich version of the game that realizes all of our initial design goals, including player animations, multiple levels, interactive items like collectibles and hazards, enemies, and a full game loop with a title screen.

Enhancement 0: Setup for Interactive Item Objects

'Approach' → Plan phase

While each enhancement is independent, they all share a common need: a way to add interactive objects like collectibles, hazards, and a goal into our level. These objects were defined in the "items" layer in our Tiled map, but our MVP code doesn't load or use them yet.

This foundational "Enhancement 0" will set the stage for all the features to come. We will:

1. Load the `items.png` spritesheet which contains the graphics for all our interactive objects.
2. Create a reusable helper method, `setup_objects()`, to efficiently give these objects basic physics properties (like being immovable and unaffected by gravity).

Completing this step will ensure our codebase is ready for the exciting features ahead.

'Apply' → Do phase

Step 1: Preload the Items Spritesheet

First, we need to load the graphics for our new objects. This will load the `items.png` file, which contains the images for our goal, hazards, and collectibles.

`src/ → Level1.js → preload()`

```
//preload external game assets
preload(){
  this.load.path = 'assets/'; //Define file path
  this.load.tilemapTiledJSON( this.map_key, this.map_json); //Load JSON file
  const tile_size = {frameWidth: 32, frameHeight: 32}
  this.load.spritesheet('tiles', 'tiles.png', tile_size); //Load tile spritesheet
  this.load.spritesheet('items', 'items.png', tile_size); //Load items spritesheet
  this.load.image( 'player', 'player.png'); //load player image
}
```

Step 2: Create a Reusable Helper Method

All of our static items from the Tiled map (like the goal and collectibles) will need the same basic physics setup. To avoid repeating code, we will create a single, reusable helper function. This method takes an array of game objects and configures them to be static, non-falling physical objects in our world.

`src/ → Level1.js → setup_objects()`

```
setup_objects(objGroup){
  for(let obj of objGroup) {
    this.physics.add.existing(obj);
    obj.body.immovable = true;
    obj.body.allowGravity = false;
  }
}
```

'Assess' → Test phase

Testing:

Since we have only loaded new assets and created a helper function without actually *using* them yet, a successful test is one where the game runs without any new errors.

Expected Outcome:

Launch the game. It should look and play **exactly the same** as it did at the end of the MVP. The player can still run, jump, and fall into pits. No new objects should appear on the screen yet.

If the game runs without crashing, you have successfully set the foundation for all the enhancements to come

Enhancement 1: Add Goal to Scene & More Levels

'Approach' → Plan phase

In this enhancement, we will add a goal to our level. When the player touches the goal, they will be transported to the next stage.

This also gives us the perfect opportunity to demonstrate one of the most powerful features of Object-Oriented Programming: **Inheritance**. We will create a `Level2` class that **extends** our `Level1` class. This means `Level2` will automatically get all of `Level1`'s functionality (player creation, physics, camera, etc.) for free, allowing us to create a new level with very little code.

Milestones:

1. Add a goal object to `Level1` and draw it in the game.
2. Create the `Level2.js` class using inheritance and wire up the scene transitions to the goal.

'Apply' → Do phase

Milestone 1 - Draw Goal Tile into Scene

Milestone 1 -- Goal:

Use the tiled json data to locate where to place the goal in the stage and which image to use to draw it.

Step 1: Invoke the `create_goal` Helper Method

To keep our `create()` method organized, we will add a call to a new helper method responsible for creating the goal. Add the following line to the `create()` method in `Level1.js`

src/ → *Level1.js* → *create()*

```
//create game data
create() {
  this.create_map();           // create level
  this.create_player();        // create player
  this.create_goal();          // create goal
  this.create_gravity();       // create gravity
  this.create_camera();        // create camera
  this.create_collisions();     // create collisions
}
```

Step 2: Implement the `create_goal` Method

This method's specific job is to find the **goal object** that we placed in our "items" object layer in Tiled and create a corresponding, visible sprite for it in our Phaser scene.

Let's break this down:

1. **goal_image**: This configuration object tells Phaser exactly what to look for. It will search the Tiled object layer named "items" for any object with the `name` property set to 'goal'.
2. **this.map.createFromObjects(...)**: This is the core function. It finds all matching objects in the Tiled data and automatically creates Phaser game objects for them, using the 'items' spritesheet and displaying frame number 3. The created objects are stored in the `this.goal` array.
3. **this.setup_objects(this.goal)**: We pass our newly created goal object(s) to our reusable helper from Enhancement 0. This gives the goal a physics body and makes it immovable so the player can't push it around.

Note: The `frame` value (3 in this case) corresponds to the tile's position in the `items.png` spritesheet, counting from zero from the top-left.

src/ → Level1.js → create_goal()

```
//Create items from object layer
create_goal(){
  const goal_image = { name: 'goal', key: 'items', frame: 3 };
  this.goal = this.map.createFromObjects('items', goal_image);
  this.setup_objects(this.goal);
}
```

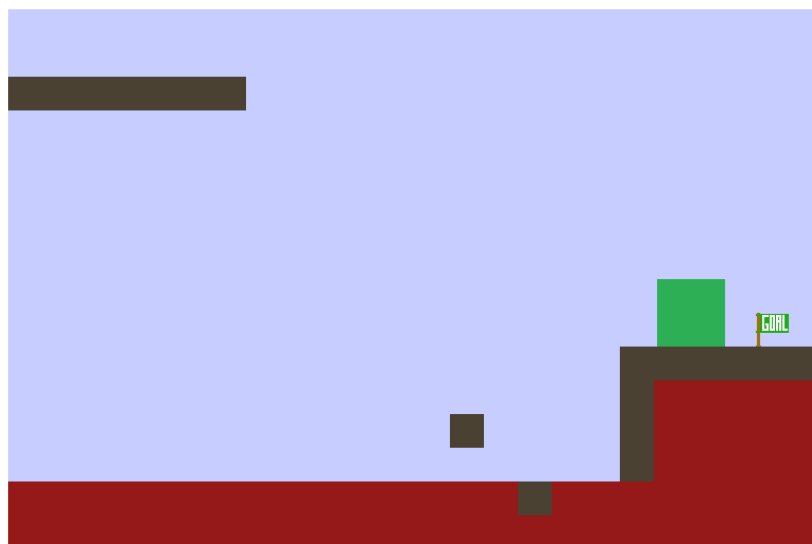
**Note: frame value should match the tile's countable position within the itemMap sprite sheet*

Milestone 1 - Testing:

1. Ensure you have completed "Enhancement 0".
2. Launch the game.
3. Navigate your player to the location where you placed the "goal" object in your Tiled map editor.

Expected Outcome:

You should see the goal sprite rendered in the game world.



Milestone 2 - Start Next Level on Collision with Goal

Milestone 2 -- Goal:

Now that our goal is visible in the scene, let's make it functional. In this milestone, we will create a second level and implement the logic to transition between the two levels when the player touches the goal.

Step 1: Create Level2 Using Inheritance

Create a new file at `src/Level2.js`. This is where we will see the power of **Inheritance**. By writing `class Level2 extends Level1`, our new scene instantly inherits *every method and property* from `Level1` (like `create_player`, `create_gravity`, etc.). We only need to define what makes `Level2` different. In the constructor, we call `super()` to register the new scene with a unique key, and then we point to the new map data for this level.

`src/ → Level2.js`

```
//level2 scene
class Level2 extends Level1 {
  constructor() {
    super('Level2');
    this.map_key = 'map2';
    this.map_json = 'level2.json';
  }
}
```

Step 2: Override create_camera method in Level2

A child class can provide its own version of a method from its parent. This is called **overriding**. Let's override the `create_camera` method to give `Level2` a unique background color, making it visually distinct from `Level1`. Add this method inside your `Level2` class.

`src/ → Level2.js → create_camera()`

```
//Setup camera to follow player
create_camera() {
  this.cameras.main.startFollow(this.player);
  this.cameras.main.setBounds(0,0,this.map.widthInPixels, this.map.heightInPixels);
  this.cameras.main.setBackgroundColor('rgb(0, 0, 0)');
}
```

Step 3: override next_scene method in Level2

To create a game loop, the goal in `Level2` should send the player back to `Level1`. We will override the `next_scene` method to change the destination. Add this method to your `Level2` class.

`src/ → Level2.js → next_scene()`

```
//start the next level
next_scene(player,goal) {
  this.scene.start('Level1');
}
```

Step 4: Register the New Scene in game.js

Our game needs to be aware that `Level2` exists. Open `src/game.js` and add `Level2` to the `scene` array in your configuration object.

`src/` → `game.js`

```
const config = new Object();

config.type = Phaser.CANVAS;           //HTML Rendering API
config.width = 32 * 24;                 //32px/tile * 8 tiles/screen
config.height = 32 * 16;                //32px/tile * 8 tiles/screen
config.pixelArt = true;                 //optimized for pixel art
config.scene = [ Level1, Level2 ];      //Scenes in this game
config.physics = { default: 'arcade' }; //Physics: collisions & gravity

const game = new Phaser.Game(config);   //Start game with these configs
```

Step 5: Load the New Scene File in index.html

Next, open `index.html` and add a script tag to load your new `Level2.js` file. The order is important: it should be loaded after its parent class, `Level1.js`.

`src/` → `index.html`

```
<html>
  <head>
    <script src="./src/phaser.js"> </script>
    <script src="./src/Player.js"> </script>
    <script src="./src/Level1.js"> </script>
    <script src="./src/Level2.js"> </script>
    <script src="./src/game.js"> </script>
  </head>
  <body></body>
</html>
```

Step 6: Define the Scene Transition Logic in Level1

Now we go back to `Level1.js` to create the function that will be called when the player touches the goal. This function's job is to start `Level2`.

`src/` → `Level1.js` → `next_scene()`

```
//start the next level
next_scene(player,goal) {
  this.scene.start('Level2');
}
```

Step 7: Add the Player-Goal Overlap Check in Level1

Finally, let's connect the collision event to the function we just created. In `Level1.js`, update your `create_collisions()` method to add a physics **overlap** check.

src/ → Level1.js → create_collisions()

```
//setups collisions with physics
create_collisions(){
  this.physics.add.collider( this.player, this.groundLayer );
  this.physics.add.overlap(this.player,this.goal,this.next_scene,null,this);
}
```

Milestone 2 -- Testing:

1. Make sure you have `level2.json` and any other required assets for it in your `assets` folder.
2. Launch the game. Play through Level 1 until you reach the goal.
3. When your player touches the goal, the scene should change to Level 2 (you'll notice the background is now black).
4. Play through Level 2 and touch its goal. The scene should loop back to Level 1.

Expected Outcome: You should be able to transition seamlessly between the two levels by touching the goal in each one.

'Assess' → Test phase

Now that all the steps for both milestones are complete, let's perform a full playtest to confirm that our goal, the new level, and the scene transitions all work together seamlessly.

Testing Instructions:

1. **Prerequisites:** Ensure you have the `level2.json` and any other required assets for the second level in your `assets` folder.
2. **Launch the game.**
3. Play through **Level 1** and navigate your player to the goal object.
4. After transitioning, play through **Level 2** and navigate to its goal.

Expected Outcome:

- When the player touches the goal in Level 1, the scene should immediately switch to Level 2. You will see the level layout change and the background will turn black, confirming the transition was successful.
- When the player touches the goal in Level 2, the game should loop back, and Level 1 should restart.
- The entire experience should be a seamless, continuous loop between the two levels.

Enhancement 2: Hazards -- Deadly Tiles

'Approach' → Plan phase

To add challenge, we need to introduce hazards -- stationary tiles like spikes that will defeat the player on contact.

We will accomplish this by creating new objects from our "items" object layer in Tiled. We will then expand our `game_over()` method to handle not only falling off the map, but also colliding with these new deadly objects. This will make our `game_over()` function a versatile, reusable tool for any lose condition we add in the future.

Milestones:

1. Add a single type of hazard and update the `game_over()` function to make it deadly.
2. Demonstrate the system's scalability by adding a second type of hazard with minimal new code.

'Apply' → Do phase

Milestone 1 - Add First Hazard

Milestone 1 -- Goal:

To add a single type of hazard (e.g., spikes) to the level. Touching this hazard should restart the scene.

Step 1: Invoke the `create_hazards` Helper Method

First, let's add a call to a new helper method in `Level1.js` that will be responsible for creating all hazard objects.

src/ → *Level1.js* → *create()*

```
//create game data
create() {
  this.create_map();           // create level
  this.create_player();        // create player
  this.create_hazards();       // create hazards
  this.create_gravity();       // create gravity
  this.create_camera();        // create camera
  this.create_collisions();     // create collisions
}
```

Step 2: Implement the `create_hazards` Method

Now, create the `create_hazards()` method. This method's job is to read the object layer from our Tiled map, find any objects we've placed with the name 'hazard1', and create corresponding, visible sprites for them in our Phaser scene. We then use our `setup_objects()` helper to make them static physics objects.

src/ → Level1.js → create_hazards()

```
//Create hazard items from object layer
create_hazards(){
  const hazard1_image = { name: 'hazard1', key: 'items', frame: 2 };
  this.group_hazard1 = this.map.createFromObjects('items', hazard1_image);
  this.setup_objects(this.group_hazard1);
}
```

**Note: frame value should match the tile's countable position within the itemMap sprite sheet*

Step 3: Make the Hazard Deadly

To make the hazard dangerous, we need to set up a physics overlap. Update your `create_collisions()` method to add a new overlap check between the player and our new hazard group. Notice that we are using `this.game_over` as the callback function.

src/ → Level1.js → create_collisions()

```
//setups collisions with physics
create_collisions(){
  this.physics.add.collider( this.player, this.groundLayer );
  this.physics.add.overlap(this.player,this.group_hazard1,this.game_over,null,this);
}
```

Step 4: Refactor game_over() to Handle Multiple Conditions

Our `game_over()` method now needs to handle two different scenarios: falling off the map, or touching a hazard. When a physics overlap callback is triggered, Phaser automatically passes the two colliding objects as arguments. We will update our function to accept these arguments.

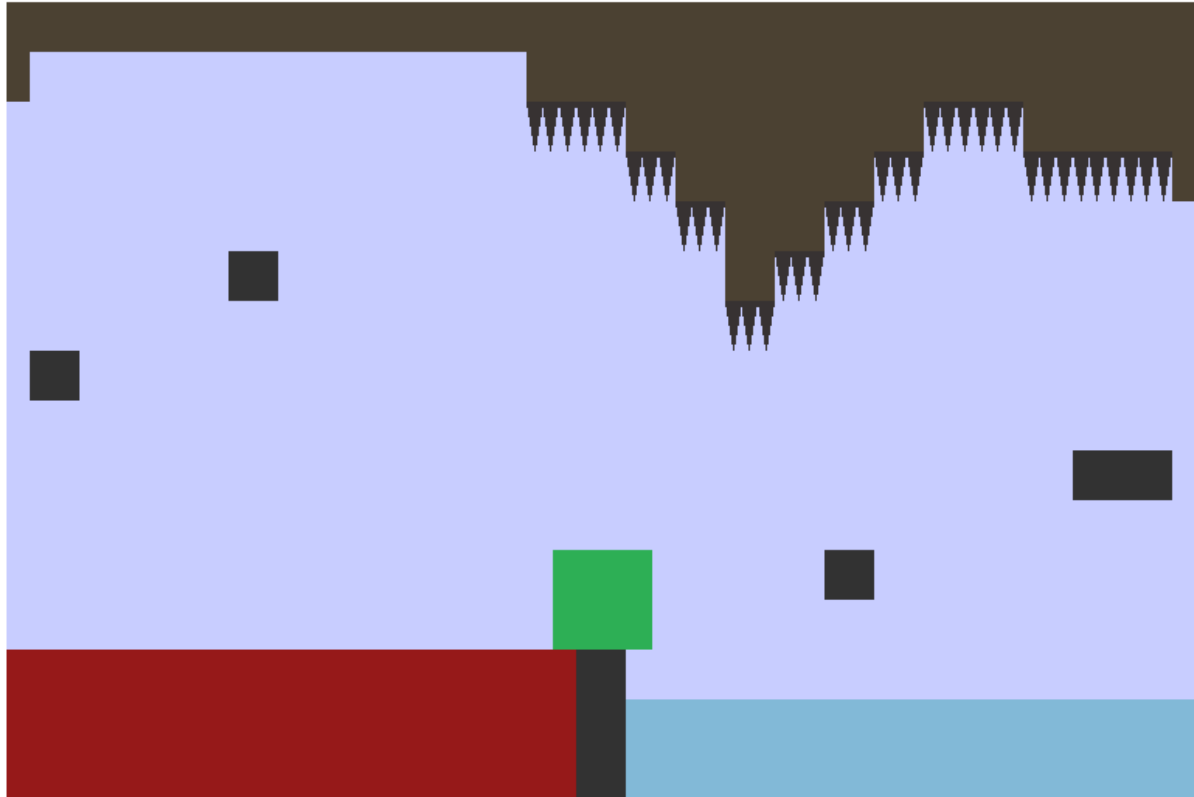
- **player = null, hazard = null:** We set default values for the parameters. When we call `game_over()` from `update()`, no arguments are passed, so `hazard` will be `null`. When the overlap triggers it, Phaser passes the player and the hazard object, so `hazard` will *not* be `null`.
- **if (hazard !== null):** This check allows us to trigger a restart only if the function was called by the hazard overlap.

src/ → Level1.js → game_over()

```
//check player lose conditions
game_over(player=null, hazard=null){
  if (this.player.y > this.map.heightInPixels) { //check if player is lower than level
    this.scene.restart();
  }
  if (hazard !== null) { //check if player is touching hazard
    this.scene.restart();
  }
}
```

Milestone 1 -- Testing:

- Launch the game and navigate your player to one of the 'hazard1' tiles.
- **Expected Outcome:** The moment the player touches the hazard, the scene should restart.



Milestone 2 - Add Second Hazard

Milestone 2 -- Goal:

To demonstrate how easily our system can be expanded by adding a second, different type of hazard.

Step 1: Create the Second Hazard in create_hazards

Refactor your `create_hazards()` method to also create objects for 'hazard2'.

Level1.js → create_hazards()

```
//Create hazard items from object layer
create_hazards(){
  const hazard1_image = { name: 'hazard1', key: 'items', frame: 2 };
  this.group_hazard1 = this.map.createFromObjects('items', hazard1_image);
  this.setup_objects(this.group_hazard1);

  const hazard2_image = { name: 'hazard2', key: 'items', frame: 1 };
  this.group_hazard2 = this.map.createFromObjects('items', hazard2_image);
  this.setup_objects(this.group_hazard2);
}
```

**Note: frame value should match the tile's countable position within the itemMap sprite sheet*

Step 2: Add the Collision Check for the Second Hazard

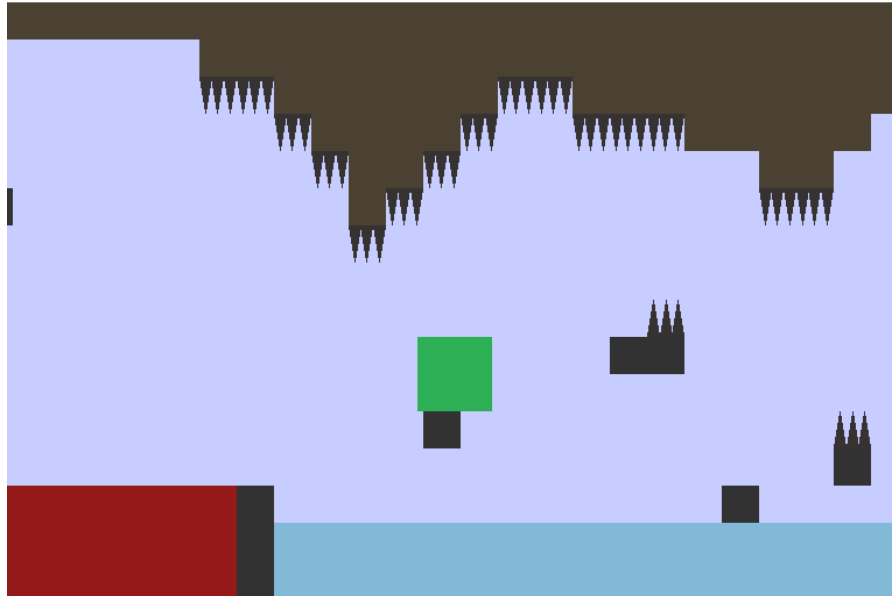
Refactor your `create_collisions()` method to add one more overlap rule for our new `group_hazard2`.

src/ → Level1.js → create_collisions()

```
//setups collisions with physics
create_collisions(){
  this.physics.add.collider( this.player, this.groundLayer );
  this.physics.add.overlap(this.player,this.group_hazard1,this.game_over,null,this);
  this.physics.add.overlap(this.player,this.group_hazard2,this.game_over,null,this);
}
```

Milestone 2 -- Testing:

- Launch the game and test both types of hazards.
- **Expected Outcome:** Touching *any* of the 'hazard1' or 'hazard2' tiles should now cause the game to restart.

**'Assess' → Test phase****Testing:**

Confirm that all lose conditions work as expected:

1. Falling off the bottom of the map restarts the level.
2. Touching a 'hazard1' (e.g., spike) restarts the level.
3. Touching a 'hazard2' (e.g., water) restarts the level.

What You've Accomplished:

You have successfully added challenge to your game by creating deadly hazards. More importantly, you have refactored a core function (`game_over`) to be more versatile, allowing it to handle multiple, different game-ending scenarios with clean, reusable code.

Enhancement 3: Hazards -- Falling Tiles

'Approach' → Plan phase

Let's add a classic platformer trap: a tile that appears solid but falls away after the player steps on it. This creates a more dynamic and surprising challenge.

The core of this mechanic is changing an object's state. The falling tile will start as a static, immovable platform. When the player collides with it, we will "turn on" gravity for that specific tile, causing it to drop. We'll also need to refine our player's jump logic to ensure they can leap off the platform as it begins to fall.

'Apply' → Do phase

Step 1: Invoke the create_falling_tiles Helper Method

First, let's add a call to a new helper method in `Level1.js` that will be responsible for creating falling tile objects.

src/ → *Level1.js* → *create()*

```
//create game data
create() {
  this.create_map();           // create level
  this.create_player();        // create player
  this.create_falling_tiles(); // create falling tile hazards
  this.create_gravity();       // create gravity
  this.create_camera();        // create camera
  this.create_collisions();     // create collisions
}
```

Step 2: Create the Falling Tile Objects

Now, implement the `create_falling_tiles()` method. This function finds all objects named 'fall' in the "items" layer and creates them as **initially static and immovable** platforms.

Here, we're doing something clever: even though we are creating an object from the **"items" object layer**, we are telling Phaser to use a graphic from the **spritesheet** for its appearance. This shows how flexible the system is -- you can place objects in one layer in Tiled but assign them graphics from any loaded spritesheet in your code.

src/ → *Level1.js* → *create_falling_tiles()*

```
//Create falling tile items from object layer
create_falling_tiles() {
  const fall_image = {name:'fall', key:'tiles', frame: 3 };
  this.group_fall = this.map.createFromObjects( 'items', fall_image);
  this.setup_objects(this.group_fall);
}
```

**Note: frame value should match the tile's countable position within the itemMap sprite sheet*

Step 3: Create the Collision Trigger

Next, we need to define the interaction that happens when the player touches one of these tiles. In `Level1.js`, update your `create_collisions()` method to add a new **collider**. We use a **collider** (not an **overlap**) because the player needs to be able to stand on the tile.

src/ → *Level1.js* → *create_collisions()*

```
//setups collisions with physics
create_collisions(){
  this.physics.add.collider( this.player, this.groundLayer );
  this.physics.add.collider(this.player,this.group_fall,this.add_gravity,null,this);
}
```

Step 4: Implement the add_gravity Callback Method

When the player touches a falling tile, this method is called, and the specific `hazard` tile that was touched is passed as an argument. Our `setup_objects()` helper explicitly set `allowGravity` to `false`. This callback's job is to reverse that for the single tile that was touched, making it subject to the world's gravity. However this line: `hazard.body.gravity.y = -1` is a physics trick that applies a tiny upward force to the tile. This makes the tile fall **just a bit slower** than the player, ensuring the player's feet stay firmly in contact. This guarantees they can reliably jump away as the platform drops.

src/ → *Level1.js* → *add_gravity()*

```
//check player lose conditions
add_gravity(player, hazard){
  hazard.body.gravity.y = -1;
  hazard.body.allowGravity = true;
}
```

Step 5: Refine the Player's Jump Logic

- `this.body.onFloor()`: This is a specific Phaser Arcade Physics helper that is very reliable for checking if the player is standing on a tilemap layer that it has a collider with. This covers our main ground and platforms.
- `this.body.touching.down`: This is a more general physics check. It becomes `true` if the bottom of the player's body is physically touching *any other physics body* below it. This is what reliably detects contact with our dynamic falling tiles.

src/ → Player.js → move()

```

//move player
move() {
  //verify that player has a physics body to move (for multilevel)
  if (this.body === undefined)
    return;

  // reset velocity
  this.body.velocity.x = 0;
  // take care of character movement
  if ( this.arrowKeys.up.isDown && ( this.body.onFloor() || this.body.blocked.down) ) {
    this.body.velocity.y = -this.speed * 2;
  }
  if ( this.arrow_keys.left.isDown ) {
    this.body.velocity.x = -this.speed;
  }
  if ( this.arrow_keys.right.isDown ) {
    this.body.velocity.x = this.speed;
  }
}

```

'Assess' → Test phase

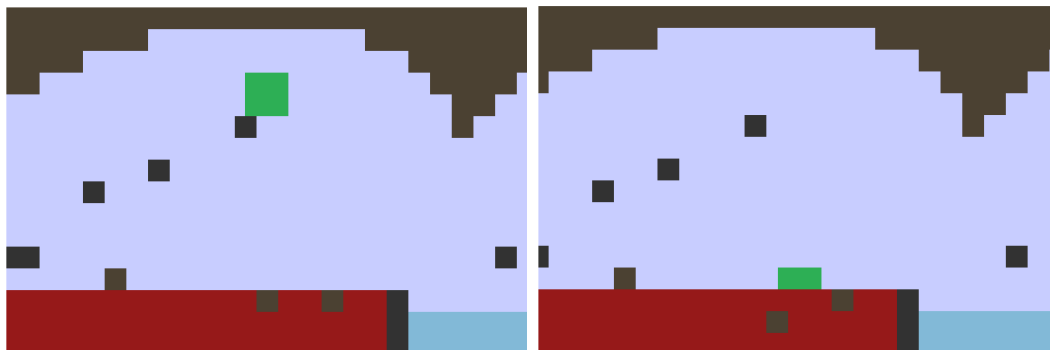
Testing:

1. Launch the game and navigate your player to one of the falling tiles.
2. Jump onto the tile.

Expected Outcome:

- Initially, the tile should be solid, and you should be able to stand on it.
- The moment your player lands on the tile, it should immediately start to fall.
- You should be able to perform a jump off the tile as it begins to drop.
- The tile will continue to fall off the bottom of the screen.
-

Testing: When the player touches the hazard, it drops down by gravity.



The tile in the red lava looks safe

But the Player falls when they jump on it

Enhancement 4: Collectables

'Approach' → Plan phase

A classic way to guide the player and add interactivity is by scattering collectable items, like coins, throughout the stage.

Goal: We will add collectable coins to our level. These coins will be created from object data in our Tiled map. When the player touches a coin, it will disappear. This enhancement will also serve as a foundation for a future scoring system. We will follow the same successful pattern: create objects from the map data and use a physics `overlap` to trigger an action.

'Apply' → Do phase

Step 1: Invoke the `create_collectables` Helper Method

To keep our main `create()` method organized, we'll start by adding a call to a new helper method that will be responsible for creating all the collectable objects.

`src/ → Level1.js → create()`

```
//create game data
create() {
  this.create_map();           // create level
  this.create_player();        // create player
  this.create_collectables();  // create collectables
  this.create_gravity();       // create gravity
  this.create_camera();        // create camera
  this.create_collisions();    // create collisions
}
```

Step 2: Implement the `create_collectables` Method

This function's job is to find all the objects we named 'collect' in our Tiled map and create corresponding, visible sprites for them in our game.

1. `collect_data`: This object tells Phaser to search the "items" layer in Tiled for any object with the `name` property set to 'collect'.
2. `this.map.createFromObjects(...)`: This creates Phaser sprites for all matching objects, using the 'items' spritesheet and displaying frame number 0 (the coin image).
3. `this.setup_objects(...)`: We use our reusable helper from Enhancement 0 to give the coins a static physics body so they don't fall due to gravity.

src/ → Level1.js → create_collectables()

```
//Create items from object layer
create_collectables(){
  const collect_data = { name: 'collect', key: 'items', frame: 0 };
  this.group_collect = this.map.createFromObjects('items', collect_data);
  this.setup_objects(this.group_collect);
}
```

**Note: frame value should match the tile's countable position within the itemMap sprite sheet*

Step 3: Create the Collectable Callback Method

When an overlap occurs, the `collect` object will be destroyed.

Level1.js → take_collectable()

```
//pick up coins - scoring logic would go in this method too
take_collectable( player, collect ) {
  collect.destroy();
}
```

Step 4: Add the Player-Collectable Overlap

Finally, we need to connect the player and the coins with a physics rule. In `Level1.js`, update your `create_collisions()` method to add a new overlap check.

We use an **overlap** instead of a collider because we want the player to pass through the coins while collecting them, rather than being blocked by them as if they were a solid wall.

Level1.js → create_collisions()

```
//setups collisions with physics
create_collisions(){
  this.physics.add.collider( this.player, this.groundLayer );
  this.physics.add.overlap( this.player, this.group_collect, this.take_collectable, null, this );
}
```

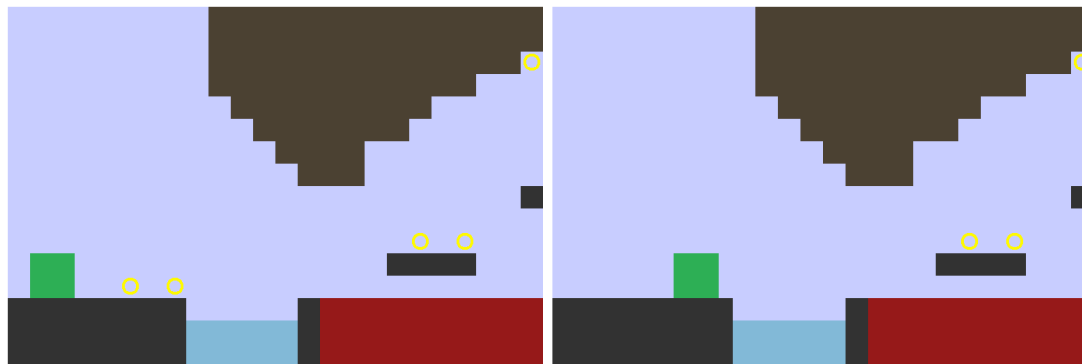
'Assess' → Test phase

Testing:

1. Ensure you have completed "Enhancement 0" so the `items.png` spritesheet is loaded and the `setup_objects()` method exists.
2. Launch the game.
3. Navigate your player around the level to the locations where you placed the 'collect' objects in Tiled.

Expected Outcome:

When the player sprite touches a coin, the coin should instantly disappear. The player should be able to pass right through the space where the coin was without stopping.



Collectables appear in the level

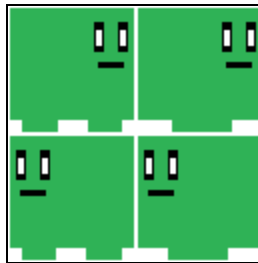
They disappear as the player touches them

Enhancement 5: Animations

'Approach' → Plan phase

Our player can move, but they look static while doing so. To bring our character to life, we will add a walk animation that plays whenever the player moves left or right.

Goal: Animate the player character when moving.



To do this, we will use Phaser's global **Animation Manager**. The process involves three key parts:

1. **Load a Spritesheet:** Replace our static `player.png` with a `player-walk.png` spritesheet that contains the individual frames of the animation.
2. **Register Animations:** In our `Level1` scene, we will define the animations once (e.g., create a "walk-left" and a "walk-right" animation) and give them unique keys.
3. **Play Animations:** In our `Player` class, we will call the `play()` method with the appropriate key based on the player's movement direction.

'Apply' → Do phase

Step 1: Replace the Player Image with a Spritesheet

First, we need to load the image containing our animation frames. In `Level1.js`, refactor the `preload()` method.

Comment out or delete the line that loads the static `player.png` and replace it with the code to load the `player-walk.png` spritesheet.

src/ → *Level1.js* → *preload()*

```
//preload external game assets
preload() {
  this.load.path = 'assets/'; //Define file path
  this.load.tilemapTiledJSON( this.map_key, this.map_json); //Load JSON file

  const tile_size = {frameWidth: 32, frameHeight: 32} //tile size props
  this.load.spritesheet('tiles', 'tiles.png', tile_size); //load tiles spritesheet
  this.load.spritesheet('items', 'items.png', tile_size); //load items spritesheet

  //this.load.image('player', 'player.png'); //load player image
  const player_size = {frameWidth: 64, frameHeight: 64} //player size props
  this.load.spritesheet( 'player-walk', 'player-walk.png', player_size ); //load player spritesheet
}
```

Step 2: Invoke the create_animations Helper Method

Next, we need a place to define our animations. In the `create()` method of `Level1.js`, add a call to a new helper, `create_animations()`. It's important to call this **before** `create_player()` so that the animations are registered and ready to be used as soon as the player is created.

src/ → *Level1.js* → *create()*

```
//create game data
create() {
  this.create_map();           // create level
  this.create_animations();    //create animations
  this.create_player();        // create player
  this.create_gravity();       // create gravity
  this.create_camera();        // create camera
  this.create_collisions();     // create collisions
}
```

Step 3: Register the Player Animations into Animation Manager

Implement `create_animations()` in `Level1` and register animations with the Scene's Animation Manager (`this.anims`). Build a small config object with the standard fields -- `key` (animation id), `frames` (ordered array of { `key: <textureKey>` }), `frameRate` (frames per second) -- then pass it to `this.anims.create(config)`. This defines the animation once globally; later, sprites just call `this.anims.play(key, true)` to use it without recreating it. We also safe-guard against adding the same animation key if it already exists in the animation manager.

src/ → *Level1.js* → *create_animations*

```
//create animations
create_animations(){
  if ( !this.anims.exists('player-left') ){
    const left_animation = new Object();
    left_animation.key = 'player-left';
    left_animation.frames = this.anims.generateFrameNumbers('player-walk', {start:2, end:3});
    left_animation.frameRate = 6;
    this.anims.create(left_animation);
  }
  if ( !this.anims.exists('player-right') ){
    const right_animation = new Object();
    right_animation.key = 'player-right';
    right_animation.frames = this.anims.generateFrameNumbers('player-walk', {start:0, end:1});
    right_animation.frameRate = 6;
    this.anims.create(right_animation);
  }
}
```

**Note: start/end values should match the tile's countable start/end position within the player sprite sheet*

Step 4: Update the Player constructor to Use the Spritesheet

In `Player.js`, we need to update the constructor. Change the texture key in the `super()` call from `'player'` to `'player-walk'` to match the new spritesheet we loaded. Then default the state of sprite to right-facing.

src/ → Player.js → constructor

```
constructor(scene) {
  super(scene, 0, 0, 'player-walk', 0);

  const start = scene.map.findObject('items', obj => obj.name === 'player' );
  this.x = start.x;
  this.y = start.y ;
  this.setOrigin(0.5,1);
  this.depth = -1;
  this.speed = 200;

  scene.add.existing(this);
  scene.physics.add.existing(this);
  this.arrow_keys = scene.input.keyboard.addKeys('up,left,right');
  this.play('player-right', false);
}
```

Step 5: Play the Appropriate Animation When the Player Moves

Finally, let's play the animations. In the `Player.js` file, update the `move()` method. Add a call to `this.play()` inside the `if` conditions for moving left and right.

src/ → Player.js → constructor

```
//move player
move() {
  //verify that player has a physics body to move (for multilevel)
  if (this.body === undefined)
    return;

  // reset velocity
  this.body.velocity.x = 0;
  // take care of character movement
  if ( this.arrow_keys.up.isDown && ( this.body.onFloor() || this.body.blocked.down) ) {
    this.body.velocity.y = -this.speed * 2;
  }
  if ( this.arrow_keys.left.isDown ) {
    this.body.velocity.x = -this.speed;
    this.play('player-left', true); //play animation
  }
  if ( this.arrow_keys.right.isDown ) {
    this.body.velocity.x = this.speed;
    this.play('player-right', true); //play animation
  }
}
```

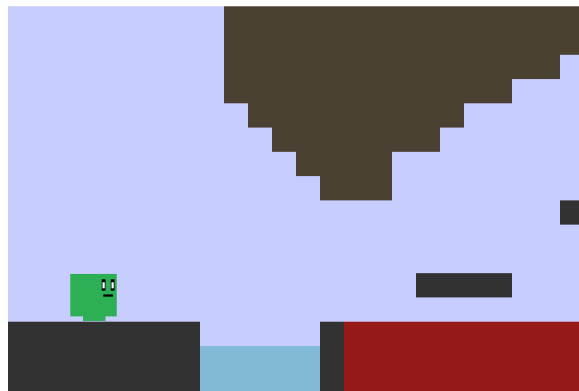
'Assess' → Test phase

Testing:

1. Launch the game.
2. Press the left and right arrow keys to move the player.

Expected Outcome:

- The player should now animate when moving.
- When you move left, the "player-left" walk cycle should play.
- When you move right, the "player-right" walk cycle should play.
- When the player is standing still or jumping, they should remain on the last frame of the animation that was played.



When player goes right



When player goes left

Enhancement 6: Enemies

'Approach' → Plan phase

A platformer needs adversaries. In this enhancement, we will add enemies to our level. These enemies will move back and forth, creating a dynamic obstacle for the player.

We will build this feature across three milestones:

1. **Add the Enemy:** We will create a new `Enemy` class, spawn enemies from our Tiled map data, and make them collide with the world and the player.
2. **Respawn Enemies:** We will add logic to make enemies respawn if they fall off the stage, ensuring the level remains challenging.
3. **Kill Enemies:** We will implement the classic platformer mechanic of defeating an enemy by jumping on its head.

'Apply' → Do phase

Milestone 1 -- Add the Enemy

Goal:

Add multiple, moving enemies into the level that will restart the game if the player touches them from the side.

Step 1: Create the Enemy.js Class

First, create a new file at `src/Enemy.js`. This class will define the behavior for a single enemy. The constructor sets up its physics properties to make it move.

- `body.velocity.x = -this.speed`: This immediately starts the enemy moving to the left.
- `body.bounce.x = 1`: This is a physics property. A bounce value of 1 means the object will perfectly reverse its velocity when it collides with another object on its X-axis (horizontally). This is what makes our enemy automatically turn around at walls.

`src/` → `Enemy.js`

```
class Enemy extends Phaser.Physics.Arcade.Sprite {
  constructor(scene, start, speed=120){
    super(scene, start.x, start.y, 'enemy');

    this.start = start;
    this.depth = 1;
    this.speed = speed;

    scene.add.existing(this);
    scene.physics.add.existing(this);
    this.body.velocity.x = -this.speed;
    this.body.bounce.x = 1;
  }
}
```


Step 2: Preload the Enemy Image

In `Level1.js`, update your `preload()` method to load the `enemy.png` image.

Level1.js → preload()

```
//preload external game assets
preload() {
  this.load.path = 'assets/'; //Define file path
  this.load.tilemapTiledJSON( this.map_key, this.map_json); //Load JSON file

  const tile_size = {frameWidth: 32, frameHeight: 32} //tile size props
  this.load.spritesheet('tiles', 'tiles.png', tile_size); //load tiles spritesheet
  this.load.spritesheet('items', 'items.png', tile_size); //load items spritesheet

  this.load.image( 'player', 'player.png' ); //load player image
  this.load.image( 'enemy', 'enemy.png' ); //load enemy image
}
```

Step 3: Invoke create_enemies helper method

Following our organized pattern, we will add a call to a new helper method, `create_enemies()`, from within our main `create()` method in `Level1.js`. This keeps our main setup function clean and readable, showing that creating enemies is part of the level's setup process.

Level1.js → create()

```
//create game data
create() {
  this.create_map(); // create level
  this.create_player(); // create player
  this.create_enemies(); // create enemies
  this.create_gravity(); // create gravity
  this.create_camera(); // create camera
  this.create_collisions(); // create collisions
}
```

Step 4: Create Enemies from Tiled Data

This function finds all objects named 'enemy' in our Tiled map, loops through them, and creates a new `Enemy` instance for each one.

Level1.js → create_enemies()

```
create_enemies() {
  this.group_enemies = [];
  let enemy_tiles = this.map.filterObjects('items', (obj) => obj.name==='enemy');
  for (let tile of enemy_tiles) {
    let enemy_config = {x: tile.x, y: tile.y};
    let enemy = new Enemy(this, enemy_config );
    this.group_enemies.push( enemy );
  }
}
```

Step 5: Add Collision Rules for Enemies

In `Level1.js`, update `create_collisions()` to make the enemies interact with the world and the player.

- The first new collider makes the enemies solid and allows them to walk on the `groundLayer`.
- The second new collider calls our versatile `game_over` function if the player touches an enemy, restarting the level.

Level1.js → create_collisions()

```
//setups collisions with physics
create_collisions(){
  this.physics.add.collider( this.player, this.groundLayer );
  this.physics.add.collider( this.group_enemies, this.groundLayer );
  this.physics.add.collider( this.player, this.group_enemies, this.game_over, null, this );
}
```

**Note: Add colliders with groupMonsters to groupBricks and groupSpikes if they also exist in your game.*

Step 6: Load the New Enemy.js File

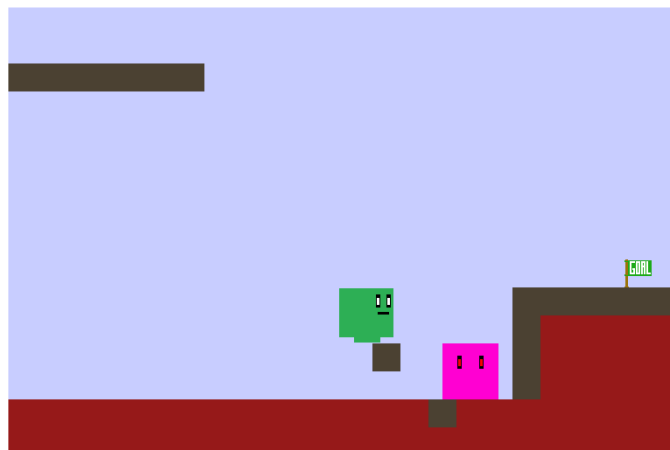
Finally, add the script tag for `Enemy.js` to your `index.html` file. It must be loaded before `Level1.js`.

index.html

```
<html>
  <head>
    <script src="./src/phaser.js">    </script>
    <script src="./src/Player.js">    </script>
    <script src="./src/Enemy.js">    </script>
    <script src="./src/Level1.js">    </script>
    <script src="./src/Level2.js">    </script>
    <script src="./src/game.js">      </script>
  </head>
  <body></body>
</html>
```

Milestone 1 - Testing:

- Launch the game. Enemies should appear at their designated spots.
- They should move back and forth, turning around automatically when they hit walls.
- If you touch an enemy from the side or from below, the level should restart.



Milestone 2 - Respawn Enemies

Goal:

Respawn an enemy to its starting position if it falls off the stage.

Step 1: Implement the Respawn Logic

We will create a system in our `update` loop to check if any enemies have fallen off the map and replace them. First, create a new helper method, `respawn_enemy()`, in `Level1.js`.

src/ → Enemy.js → respawn_enemy()

```
respawn_enemy(enemy_idx) {  
  const enemy = this.group_enemies[enemy_idx];  
  if(enemy.y > this.map.heightInPixels) {  
    const new_enemy = new Enemy(this, enemy.start, enemy.speed)  
    this.group_enemies.splice(enemy_idx, 1); // remove old  
    this.group_enemies.push( new_enemy ); // add replacement  
  }  
}
```

Step 2: Create the Enemy Update Loop

Now, create a new helper method, `update_enemies()`, that will loop through all active enemies and call the respawn check on each one.

src/ → Level1.js → update_enemies()

```
update_enemies() {  
  for (let enemy_idx in this.group_enemies){  
    this.respawn_enemy(enemy_idx);  
  }  
}
```

Step 3: Invoke the Enemy Update Loop

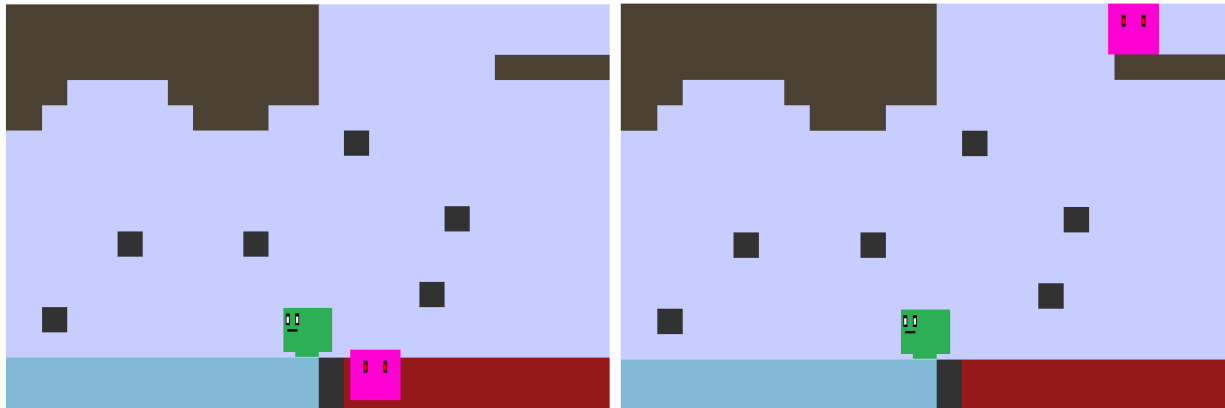
Finally, call `this.update_enemies()` from the main `update()` method in `Level1`.

src/ → Level1.js → update()

```
//Update game data  
update(){  
  this.update_player();  
  this.update_enemies();  
  this.game_over();  
}
```

Milestone 2 - Testing:

- Find an enemy that falls into a pit.
- Once the enemy falls off the bottom of the map, a new one should reappear at its original starting position.



Enemy falls down into the void

New enemy respawns to replace it

Milestone 3 - Kill Enemies

Goal:

Allow the player to defeat an enemy by jumping on top of it.

Step 1: Create the attack Callback Method

In `Level1.js`, create a new method called `attack()`. This will be the callback function for our new collision rule. It checks *how* the player and enemy collided.

src/ → Level1.js → attack()

```
attack(player, enemy){  
  if (player.body.touching.down)  
    enemy.destroy()  
  else  
    this.game_over(player, enemy);  
}
```

Step 2: Update the Collision Rule

In `create_collisions()`, we will now change the collider between the player and enemies. Instead of calling `game_over` directly, it will now call our new, more intelligent `attack` function

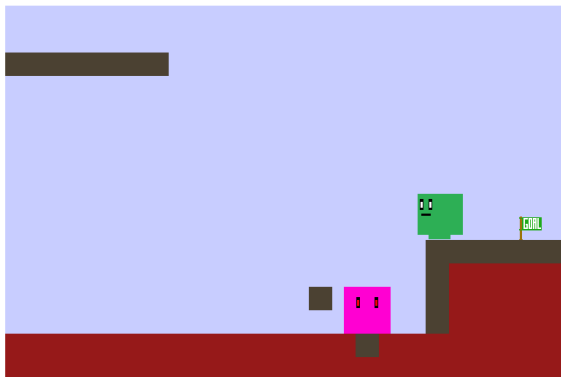
src/ → Level1.js → create_collisions()

```
//setups collisions with physics  
create_collisions(){  
  this.physics.add.collider( this.player, this.groundLayer );  
  this.physics.add.collider( this.group_enemies, this.groundLayer );  
  this.physics.add.collider( this.player, this.group_enemies, this.attack, null, this )  
}
```

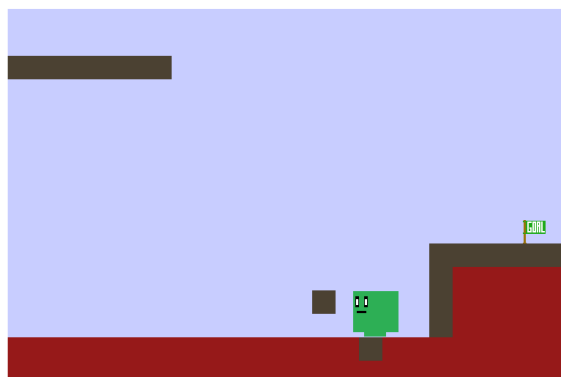
**Note: Add colliders with groupMonsters to groupBricks and groupSpikes if they also exist in your game.*

Milestone 2 -- Testing:

- Jump on top of an enemy. The enemy should disappear.
- Walk into the side of an enemy. The level should restart.



Enemy and Player



Enemy destroyed after Player hopped on it

'Assess' → Test phase

Testing:

Perform a full playtest to confirm all enemy mechanics work together.

- Enemies should patrol back and forth.
- Touching an enemy from the side is deadly.
- Jumping on top of an enemy defeats it.
- Enemies that fall off the stage should respawn.

Enhancement 7: Title Screen

'Approach' → Plan phase

A polished game needs a proper introduction and a way to exit the gameplay loop. In this enhancement, we will add a **Title Screen** to serve as a formal main menu and starting point for the player.

While we already have multiple levels, this change gives our game a proper structure with a clear beginning and an exit point from the main action. To manage the transitions between the title screen and our game levels, we will make formal use of a key Phaser system:

- **The Scene Manager:** This is the powerful tool we have been using implicitly all along (`this.scene.start`). It allows us to control the game flow by starting, stopping, and switching between different scenes. In this enhancement, we will use it to create the complete game loop: from the title screen, to the game, and back again.

'Apply' → Do phase

Milestone 1: Create the Basic Title Scene

Milestone Goal:

First, we'll create a new `TitleScene` class, display the game title and instructions, and make it the new starting point for our game.

Step 1: Create the class `TitleScene`

Create a new file at `src/TitleScene.js`. This will be a simple scene whose only job is to display text. Its constructor registers itself with the key `'title'`. The `create()` method invokes a helper method

`src/` → `TitleScene.js`

```
// src/TitleScene.js
class TitleScene extends Phaser.Scene {
  constructor() {
    super('title'); // Register scene with key 'title'
  }

  create() {
    // We will add content here in the next step
    this.create_title()
  }
}
```

Step 2: Add Title and Instruction Text

In the `TitleScene`'s `create()` method, we'll use `this.add.text()` to display our game's UI. A useful trick for centering text is to use the game's width/height and set the text's **origin** to `(0.5, 0.5)`. The origin is the text's "anchor point" on a 0-to-1 scale.

src/ → *TitleScene.js* → *create_title()*

```
create_title() {
  const width = this.game.config.width;
  const height = this.game.config.height;

  // Game Title
  let font_config = { fontSize: '48px', fill: '#FFFFFF' };
  const game_title = this.add.text(width / 2, height / 3, 'Platformer GAME', font_config);
  game_title.setOrigin(0.5);
  game_title.setResolution(1);

  // Instructions
  font_config = { fontSize: '24px', fill: '#FFFFFF', align: 'center' };
  const instruction_text = this.add.text(width / 2, height / 2, 'Arrow Keys to Move & Jump', font_config);
  instruction_text.setOrigin(0.5);
  instruction_text.setResolution(1);

  // Start prompt
  font_config = { fontSize: '24px', fill: '#FFFF00' };
  const start_text = this.add.text(width / 2, height * 2 / 3, 'Press SPACE to Start', font_config);
  start_text.setOrigin(0.5);
  start_text.setResolution(1);
}
```

Step 3: Make the Title Screen the Starting Scene

Now we need to tell Phaser to launch our new `TitleScene` first. Open `src/game.js` and modify the `scene` array in the game configuration. The first scene in this array is the one that loads on startup.

src/ → *game.js*

```
const config = new Object();

config.type = Phaser.CANVAS; //HTML Rendering API
config.width = 32 * 24; //32px/tile * 8 tiles/screen
config.height = 32 * 16; //32px/tile * 8 tiles/screen
config.pixelArt = true; //optimized for pixel art
config.scene = [ TitleScene, Level1, Level2 ]; //Scenes in this game
config.physics = { default: 'arcade' }; //Physics: collisions & gravity

const game = new Phaser.Game(config); //Start game with these configs
```

Step 4: Make the Title Screen the Starting Scene

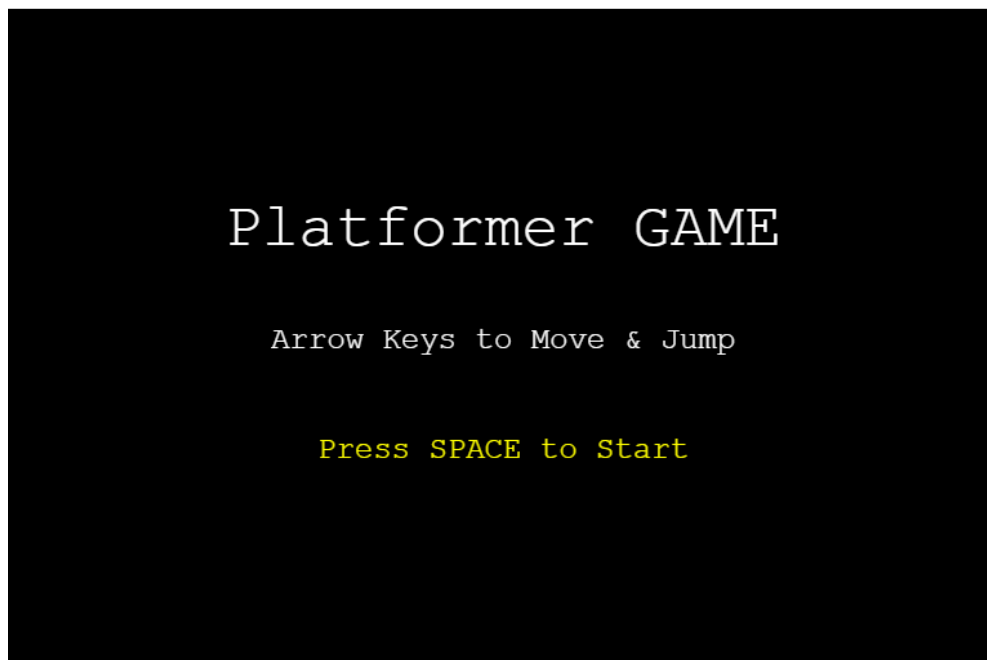
Finally, add the `TitleScene.js` script to your `index.html`, ensuring it is loaded before `game.js`.

index.html

```
<html>
  <head>
    <script src="./src/phaser.js">    </script>
    <script src="./src/Player.js">    </script>
    <script src="./src/Enemy.js">    </script>
    <script src="./src/Level1.js"> </script>
    <script src="./src/Level2.js">    </script>
    <script src="./src/TitleScene.js"> </script>
    <script src="./src/game.js">    </script>
  </head>
  <body></body>
</html>
```

(Milestone 1) Testing:

Launch the game. It should now open to your new Title Screen with the title and instructions, instead of jumping directly into the action.



Milestone 2: Implement Scene Switching

Milestone Goal:

Now we'll wire up the scene transitions: pressing `Space` on the title screen will start the game, and pressing `Escape` during gameplay will return to the title screen.

Step 1: Start the Game from the Title Screen

In `TitleScene.js`, we need to listen for a key press. We can use an event listener for this. In the `create()` method, add a listener that waits for the `SPACE` key to be pressed. The callback function will call `this.scene.start('Level1')`, which shuts down the current scene (`title`) and launches the one with the key `'Level1'`.

src/ → TitleScene.js → create()

```
create() {  
  this.create_title();  
  this.input.keyboard.on('keydown-SPACE', () => { this.scene.start('Level1'); });  
}
```

Step 2: Return to the Title Screen from the Play Scene

In `Level1.js`, we need to add a similar listener. In the `create()` method, add a listener for the `ESC` key. The callback will start the `'title'` scene.

src/ → Level1.js → create()

```
// src/Level1.js → in create()  
create() {  
  // ... (this can go anywhere in the create method)  
  this.input.keyboard.on('keydown-ESC', () => { this.scene.start('title'); });  
}
```

(Milestone 2) Testing:

Run the full flow. Start the game, you see the title. Press `Space`, the game starts. Play for a bit, then press `Escape`. You should return to the title screen. **Notice a problem:** if you got a high score, it resets! We'll fix this next.

Conclusions

Final Comments

This lab was a multifaceted journey through modern web and game development, building on foundational concepts to create a feature-rich game. You have successfully:

1. **Applied Object-Oriented JavaScript**, especially **Inheritance**, to extend a professional framework and create new game levels with minimal, reusable code.
2. **Mastered core concepts of the Phaser 3 library**, including advanced topics like the **Tilemap engine**, **Arcade Physics for platformers**, and multi-scene management.
3. **Utilized software engineering techniques** (sprints, milestones) to iteratively build a complex project from a simple MVP to a feature-rich final product.
4. **Delivered a complete and polished Platformer game**, showcasing key concepts like multi-level design, data-driven worlds using Tilemaps, and classic platformer mechanics like jumping, collecting items, and stomping enemies.

Future Improvements (*Beyond the Scope of this Lab*)

The foundation you've built is strong, but there are many exciting ways you could expand on this project. Consider trying to implement some of these features:

- **Implement a Scoring System and a HUD (Heads-Up Display)** to track collected items, score, and lives.
- **Add More Enemy Types** with different behaviors and patterns (e.g., flying or shooting enemies).
- **Expand the Player's Abilities** with moves like a double jump, wall slide, or a ranged attack.
- **Design More Complex Levels** with unique mechanics like moving platforms, switches, or keys and doors.
- **Add Boss Battles** at the end of a series of levels for a greater challenge.
- **Implement Parallax Scrolling** for background layers to create a sense of depth as the camera moves.
- **Add Sound Effects and Background Music** to bring the world to life.
- **Use the browser's `LocalStorage`** to save data (like high scores or level progress) between sessions.
- **Optimize Performance** by deactivating enemies and objects that are far off-screen.

Lab Submission

Push all of your lab files into your forked repository and close this Lab in your github issues. Mark it as complete.

Module 2 - Project:

Create your own front-end browser app with JavaScript & Canvas API / Phaser or modifies the DOM.

Project Bonus:

Showcase bonus. You can receive up to 20 bonus points if your project is outstanding and novel. I'll publish all showcase projects as a demo for future students. You should reference such projects on your resume or CV.